



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that **Arnav Santosh Sawant** of **D20A** semester **VIII**, have successfully completed necessary experiments in the **ITC801: Blockchain & DLT Lab** under my supervision in **VES Institute of Technology** during the academic year **2025-2026**.

Subject Teacher

Name: Mrs. Kajal Joseph

Signature:

Head of Department

Name: Dr. Mrs. Shalu Chopra

Signature:

Lab Teacher

Name: Kajal Joseph

Signature:



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

Lab Code	Lab Name	Credit
ITC801	Blockchain and DLT Lab	1

Programme Outcomes: The graduate will be able to:

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design / develop systems /components / processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.
PSO 1	Computing & Emerging Technology: Apply core computing and modern technologies, such as AI, Cloud Computing, IoT, cybersecurity, and data-driven tools, to build efficient and innovative solutions.
PSO 2	Professionalism & Innovation : Demonstrate ethics, teamwork, and an entrepreneurial mindset to deliver sustainable solutions for society.

Course Outcomes: Student will be able	
ITC801.1	Describe the basic concept of Blockchain and Distributed Ledger Technology.
ITC801.2	Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions
ITC801.3	Implement smart contracts in Ethereum using different development frameworks.
ITC801.4	Develop applications in permissioned Hyperledger Fabric network.
ITC801.5	Interpret different Crypto assets and Crypto currencies
ITC801.6	The use of Blockchain with AI, IoT and Cyber Security using case studies.

ITC801 Blockchain and DLT Lab													
CO	PO											PSO	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2
ITC801.1	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.2	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.3	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.4	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.5	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.6	2	3	3	3	3	3	3	3	3	2	2	3	3
Avg PO AL	2	3	3	3	3	3	3	3	3	2	2	3	3



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801 Blockchain and DLT Lab												
CO	Experiment											
	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9	Exp 10	Exp 11	Exp 12
ITC801.1	3	3	2	-	2	2	2	-	-	3	-	3
ITC801.2	2	3	3	-	2	2	3	2	-	3	-	3
ITC801.3	-	-	2	3	3	3	3	3	2	-	-	3
ITC801.4	-	-	-	-	-	-	-	-	-	-	3	3
ITC801.5	2	3	3	-	-	2	2	2	-	3	-	3
ITC801.6	-	-	-	-	-	-	-	-	-	2	-	3

INDEX

Sr. No.	List of Experiments
1	Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash
2	Create a Blockchain using Python
3	Create a Cryptocurrency using Python and perform mining in the Blockchain created.
4	Hands-on Solidity Programming Assignments for creating Smart Contracts
5	Deploying a Voting/Ballot Smart Contract
6	Creating Smart Contracts and performing transactions using Solidity and Remix IDE
7	Implement the embedding wallet (Metamask) and transaction using Solidity
8	To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

9	To implement a Private Ethereum Blockchain using Geth.
10	Explore the Cryptocurrency Landscape
11	Implementation of Hyperledger Fabric Network and Asset Management using Chaincode
12	Mini Project
12	Assignment-1
13	Assignment-2
14	Assignment-3
15	Mock Viva

Subject teacher



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment No. 01

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Roll No.	54
Name	Arnav Sawant
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Aim of the Experiment

To study and implement cryptographic hashing using the SHA-256 algorithm and to understand the construction and working of the Merkle Tree in blockchain technology by generating hashes, solving a simple Proof-of-Work puzzle, and computing the Merkle Root for a set of transactions in order to demonstrate data integrity and security in blockchain systems.

Theory:-

1. Cryptographic Hash Function in Blockchain

A **cryptographic hash function** is a mathematical algorithm that takes an input of any size and converts it into a **fixed-length output** called a *hash value* (or *digest*).

In blockchain, the most commonly used hash function is **SHA-256 (Secure Hash Algorithm – 256 bit)**.

Properties of a Cryptographic Hash Function:

1. **Deterministic** – Same input always gives same hash
2. **Fixed length output** – Always 256 bits in SHA-256
3. **Fast computation**
4. **Pre-image resistance** – Cannot reverse hash to original data
5. **Avalanche effect** – Small change → completely different hash
6. **Collision resistance** – Very hard to find two same hashes

Role in Blockchain:

- Secures transactions
- Links blocks together
- Used in mining (Proof-of-Work)
- Ensures data integrity

2. Merkle Tree (Hash Tree)

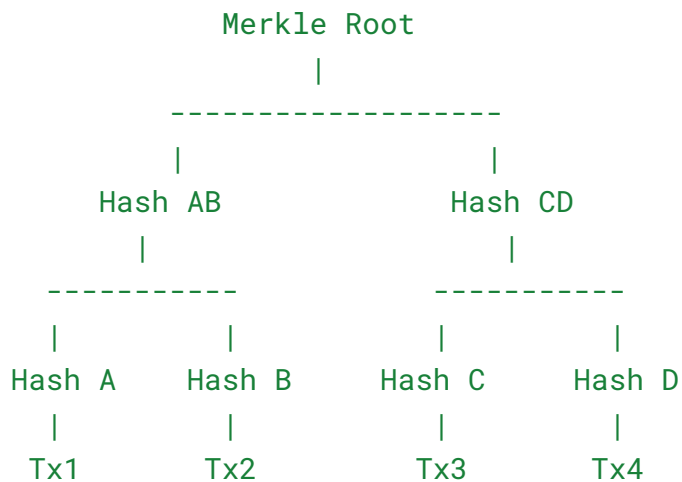
A **Merkle Tree** is a **binary tree of hashes** used to efficiently summarize and verify large sets of data.

- **Leaf nodes** → hashes of transactions
- **Non-leaf nodes** → hash of combined child hashes
- **Top node** → Merkle Root

Merkle Tree allows fast and secure verification of data.

3. Structure of a Merkle Tree

A Merkle Tree is usually a **binary tree** structure:



Components:

- **Leaf nodes** – Hash of transactions
- **Internal nodes** – Hash of two child hashes

- **Root** – Final combined hash of all transactions

If the number of transactions is odd, the **last hash is duplicated**.

4. Merkle Root

The **Merkle Root** is the **topmost hash** of the Merkle Tree.

It represents **all transactions in a block with a single hash value**.

Importance:

- Stored in the **block header**
 - If any transaction changes → Merkle root changes
 - Used for **quick verification**
-

5. Working of Merkle Tree

Step-by-step working:

1. Each transaction is hashed using SHA-256
2. Hashes are paired and concatenated
3. Each pair is hashed again
4. If odd number of hashes → duplicate last one
5. Repeat process until one hash remains
6. Final hash is the **Merkle Root**

Example:

Tx1 → H1

Name:- Arnav Sawant D20A / 52

$Tx2 \rightarrow H2$

$Tx3 \rightarrow H3$

$Tx4 \rightarrow H4$

$H1+H2 \rightarrow H12$

$H3+H4 \rightarrow H34$

$H12+H34 \rightarrow \text{Merkle Root}$

6. Benefits of Merkle Tree

- ✓ Fast verification of transactions
- ✓ Very secure and tamper-proof
- ✓ Requires less storage
- ✓ Efficient for large datasets
- ✓ Supports partial verification (SPV)
- ✓ Any small change changes the Merkle root

7. Use of Merkle Tree in Blockchain

Merkle Trees are used to:

- Store transactions inside a block
- Verify whether a transaction belongs to a block
- Maintain data integrity
- Support **lightweight (SPV) nodes**
- Reduce data transfer
- Secure block structure

In Bitcoin and Ethereum, Merkle trees are used to organize and validate transactions.

8. Use Cases of Merkle Tree

1. **Blockchain systems** (Bitcoin, Ethereum, Hyperledger)
 2. **Distributed databases**
 3. **File systems** (Git, IPFS)
 4. **Cloud storage verification**
 5. **Digital signatures**
 6. **Peer-to-peer networks**
 7. **Secure data synchronization**
 8. **Certificate transparency logs**
-

9. Code

```
import hashlib

# -----
# 1. SHA-256 Hash Generation
# -----
def sha256_hash(data):
    return hashlib.sha256(data.encode()).hexdigest()

# -----
# 2. Hash Generation with Nonce (Mining simulation)
# -----
def hash_with_nonce(data, nonce):
    text = data + str(nonce)
    return sha256_hash(text)

# -----
# 3. Proof of Work (find nonce for leading zeros)
```

```
# -----
def proof_of_work(data, difficulty):
    print("\n🔍 Mining started...")
    prefix = "0" * difficulty
    nonce = 0

    while True:
        new_hash = hash_with_nonce(data, nonce)
        if new_hash.startswith(prefix):
            return nonce, new_hash
        nonce += 1

# -----
# 4. Merkle Tree & Merkle Root
# -----
def merkle_root(transactions):
    # Step 1: hash all transactions
    hashes = [sha256_hash(tx) for tx in transactions]

    # Step 2: keep combining until one hash remains
    while len(hashes) > 1:
        if len(hashes) % 2 != 0:    # if odd, duplicate last
            hashes.append(hashes[-1])

        new_level = []
        for i in range(0, len(hashes), 2):
            combined = hashes[i] + hashes[i+1]
            new_level.append(sha256_hash(combined))

        hashes = new_level

    return hashes[0]

# -----
# Main Program
# -----
if __name__ == "__main__":
```

```
# 1. SHA-256
msg = input("Enter a string: ")
print("\nSHA-256 Hash:", sha256_hash(msg))

# 2. Hash with nonce
nonce = int(input("\nEnter a nonce: "))
print("Hash with nonce:", hash_with_nonce(msg, nonce))

# 3. Proof of Work
difficulty = int(input("\nEnter difficulty (number of leading zeros):
"))
nonce, result_hash = proof_of_work(msg, difficulty)
print("\n✅ Proof of Work Found!")
print("Nonce:", nonce)
print("Hash:", result_hash)

# 4. Merkle Tree
print("\nEnter transactions (type 'done' to finish):")
transactions = []
while True:
    tx = input()
    if tx.lower() == "done":
        break
    transactions.append(tx)

if len(transactions) > 0:
    root = merkle_root(transactions)
    print("\n🌳 Merkle Root:", root)
else:
    print("No transactions entered.")
```

10. Conclusion

Merkle Trees play a crucial role in blockchain by providing a secure, efficient, and scalable method to verify transactions. By using cryptographic hashing, Merkle Trees ensure data integrity, fast verification, and reduced storage requirements, making them essential for modern decentralized systems.

Name:- Arnav Sawant D20A / 52



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain & DLT Lab
Experiment 02

Aim: Create a Blockchain using Python

Roll No.	54
Name	Arnav Sawant
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Experiment No. 2

Aim:- Create blockchain using python

What is Blockchain?

Blockchain is a modern digital technology used to store information in a **secure, transparent, and tamper-proof way**. It works like a **shared online record book (ledger)** that keeps track of transactions.

The term **blockchain** comes from how the data is stored:

- Information is saved inside **blocks**
- Each block is connected to the **previous block**
- Together they form a **chain of blocks**

Every block contains:

- Transaction details
- Time of creation (timestamp)
- A special security code called a **cryptographic hash**

This hash links one block to another. If someone tries to change the data, the hash changes and the chain breaks, making tampering easy to detect.

Key Features of Blockchain

- Distributed across many computers (nodes)
- No single authority controls it (decentralized)
- Transactions are verified by the majority of nodes
- Once data is added, it cannot be changed (immutable)
- Highly secure and transparent

In simple words, blockchain is a **decentralized database that safely records transactions across multiple systems**.

What is a Block?

A **block** is a basic unit of storage in a blockchain.

Think of it like **one page in a ledger book** that stores transaction records.

Many transactions happen daily, and blocks help organize and store these transactions securely. Once a block is full, it gets added to the blockchain and linked with other blocks.

Components of a Block

Block Header

The block header contains important information that uniquely identifies the block. It helps maintain the order and connection between blocks.

Previous Block Hash

This stores the hash of the previous block.

It links the current block to the earlier one, forming the **chain structure** of the blockchain.

Timestamp

Records the exact **date and time** when the block was created.

It proves when the transactions were added.

Nonce

Nonce means “**number used only once.**”

- Used during mining (Proof of Work)
 - Miners keep changing the nonce to find a valid hash
 - Helps secure the block and validate it
-

Merkle Root

The Merkle Root is created using a **Merkle Tree**.

- Combines all transactions into one hash
- Acts like a digital fingerprint
- Allows quick and secure verification of transactions

Block {

Index: 2

Timestamp: 2026-01-22 10:05:00

Data: {"Sender":"Alice", "Receiver":"Bob", "Amount":50}

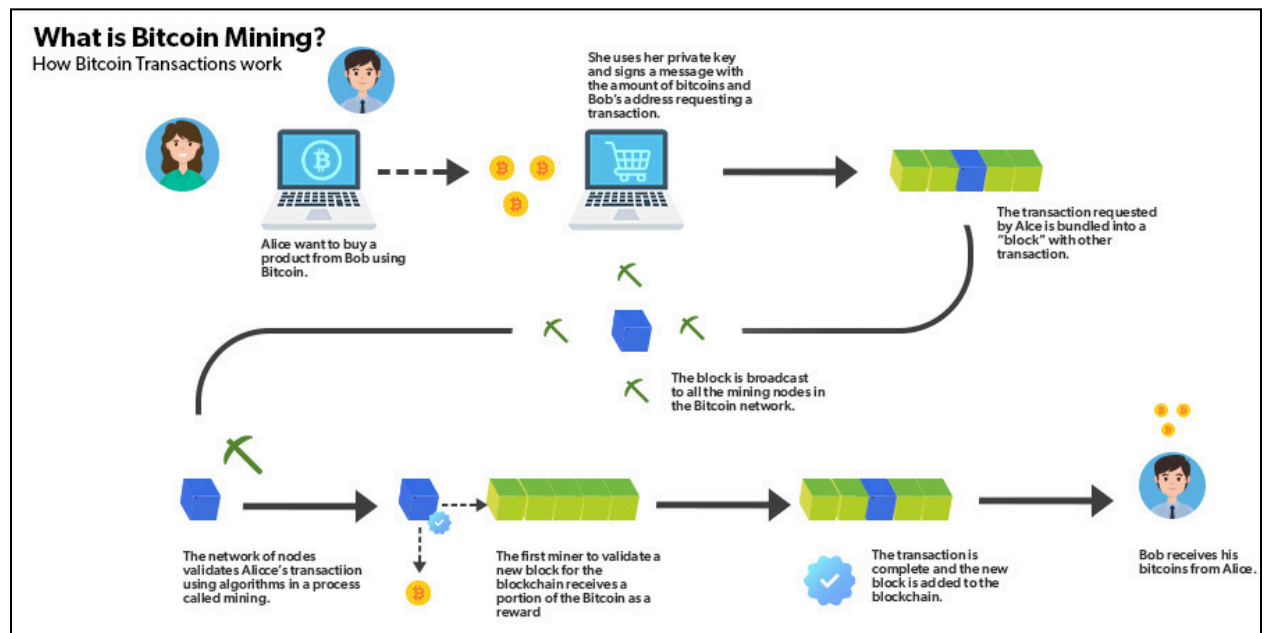
Nonce: 102345

Previous Hash: "0000a1b2c3d4e5..."

Hash: "0000f3b7d6c1e8..."

}

Process of mining



Mining is the process of adding a new block to the blockchain.

It ensures security and integrity using Proof-of-Work (PoW).

Step 1: Collect Transactions

- All pending transactions are collected into a new block.
- Example: Alice sends 50 coins to Bob.

Step 2: Create Block Header

Block header includes:

- Previous hash
- Transactions
- Timestamp
- Nonce (initially 0)

Step 3: Proof of Work (PoW)

- The miner repeatedly changes the nonce to find a hash that satisfies the difficulty requirement.
- Difficulty = number of leading zeroes required in hash (e.g., 4 leading zeroes).

SHA256(block_data + nonce) → hash starts with "0000"

- This requires computational work, hence the name Proof-of-Work.

Step 4: Validate and Broadcast

- Once a valid hash is found, the block is broadcast to all nodes in the network.
- Other nodes verify:
 1. Hash correctness
 2. Nonce validity
 3. Previous hash link

Step 5: Add to Blockchain

- If all nodes validate, the block is added to the chain.
- The miner receives a reward (e.g., cryptocurrency).

How to check validity of blocks in blockchain

Step 1: Validate Previous Hash

- Each block's previous_hash must match the hash of the previous block.

```
if Block[n].previous_hash != Hash(Block[n-1]):  
    Invalid
```

Step 2: Recalculate Current Hash

- Compute hash of current block's data and nonce.
- It must match the stored hash in the block.

```
if SHA256(Block[n].data + Block[n].nonce) != Block[n].hash:  
    Invalid
```

Step 3: Check Proof-of-Work

- Verify that the block hash satisfies the difficulty (e.g., 4 leading zeroes).

if not Block[n].hash.startswith("0000"):

Invalid

Step 4: Validate Entire Chain

- Repeat steps 1–3 for all blocks from Genesis to the latest block.
- If any block fails → blockchain is invalid.

Step 5: Validate Genesis Block

- The first block (Genesis block) is hardcoded.
- Previous hash = "0"
- Must not be altered.

```

PS C:\NRK> python -m pip install flask
Collecting flask
  Downloading flask-3.1.2-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp314-cp314-win_amd64.whl.metadata (2.8 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.5-py3-none-any.whl.metadata (4.0 kB)
Collecting colorama (from click>=8.1.3->flask)
  Downloading colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Downloading flask-3.1.2-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.1-py3-none-any.whl (108 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.3-cp314-cp314-win_amd64.whl (15 kB)
Downloading werkzeug-3.1.5-py3-none-any.whl (225 kB)

```

```

Installing collected packages: markupsafe, itsdangerous, colorama, blinker, werkzeug, jinja2, click, flask
 7/8 [flask] WARNING: The script flask.exe is installed in 'C:\Us
rs\admin\AppData\Local\Python\pythoncore-3.14-64\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-loc
tion.
Successfully installed blinker-1.9.0 click-8.3.1 colorama-0.4.6 flask-3.1.2 itsdangerous-2.2.0 jinja2-3.1.6 m
arkupsafe-3.0.3 werkzeug-3.1.5

```

blockchain.py

```

import datetime
import hashlib
import json

```

```

class Blockchain:
    def __init__(self):
        self.chain = []
        self.difficulty = 4 # 4 leading zeroes
        self.create_genesis_block()

    def create_genesis_block(self):
        genesis_block = {

```

```

        'index': 1,
        'timestamp': str(datetime.datetime.now()),
        'data': 'Genesis Block',
        'nonce': 0,
        'previous_hash': '0'
    }
    genesis_block['hash'] = self.calculate_hash(genesis_block)
    self.chain.append(genesis_block)

def calculate_hash(self, block):
    block_copy = block.copy()
    block_copy.pop('hash', None)
    encoded = json.dumps(block_copy, sort_keys=True).encode()
    return hashlib.sha256(encoded).hexdigest()

def mine_block(self, data):
    previous_block = self.chain[-1]
    nonce = 0

    while True:
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'data': data,
            'nonce': nonce,
            'previous_hash': previous_block['hash']
        }

        block_hash = self.calculate_hash(block)

        if block_hash.startswith('0' * self.difficulty):
            block['hash'] = block_hash
            self.chain.append(block)
            return block

        nonce += 1

def is_chain_valid(self):
    for i in range(1, len(self.chain)):
        current = self.chain[i]
        previous = self.chain[i - 1]

        if current['previous_hash'] != previous['hash']:
            return False

```

```

        recalculated_hash = self.calculate_hash(current)
        if recalculated_hash != current['hash']:
            return False

        if not current['hash'].startswith('0' * self.difficulty):
            return False

    return True

```

app.py

```

from flask import Flask, jsonify
from blockchain import Blockchain

app = Flask(__name__)
blockchain = Blockchain()

@app.route('/mine_block', methods=['GET'])
def mine_block():
    block = blockchain.mine_block("Some transaction data")

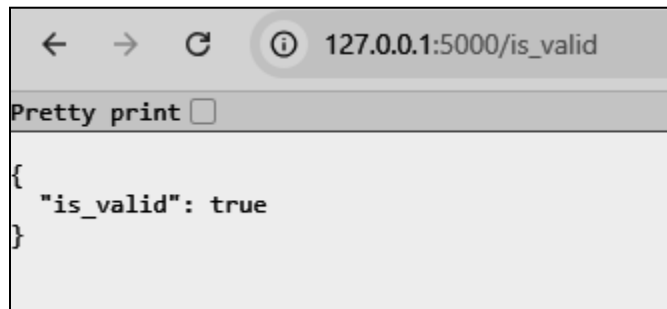
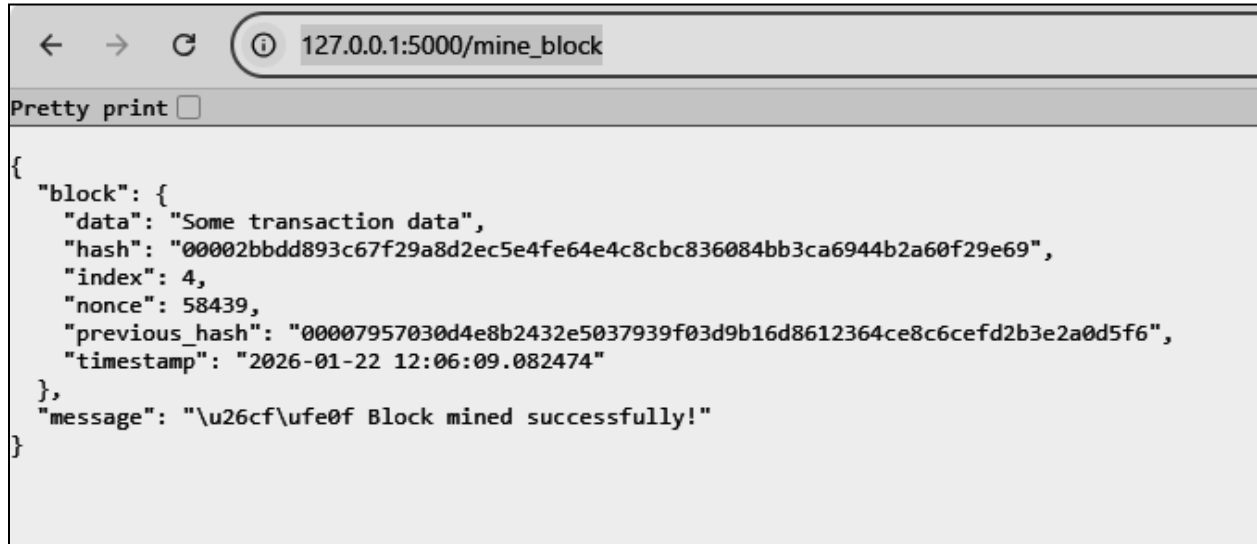
    return jsonify({
        'message': '🔨 Block mined successfully!',
        'block': block
    }), 200

@app.route('/get_chain', methods=['GET'])
def get_chain():
    return jsonify({
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }), 200

@app.route('/is_valid', methods=['GET'])
def is_valid():
    return jsonify({
        'is_valid': blockchain.is_chain_valid()
    }), 200

```

```
if __name__ == '__main__':  
    print(" Mining blockchain with 4 leading zeroes...")  
    app.run(debug=True, use_reloader=False)
```



← → ↻ ⓘ 127.0.0.1:5000/get_chain

Pretty print ☐

```
{
  "chain": [
    {
      "data": "Genesis Block",
      "hash": "d1c793d584710b37ed90d397f2f7a59cd9f741b1f424553cae1948dfeed661c4",
      "index": 1,
      "nonce": 0,
      "previous_hash": "0",
      "timestamp": "2026-01-22 12:05:29.544918"
    },
    {
      "data": "Some transaction data",
      "hash": "000030b5874b1e81cad31a1eb57cbcf69db993bbb0af36834c9742c4bea27ab5",
      "index": 2,
      "nonce": 215962,
      "previous_hash": "d1c793d584710b37ed90d397f2f7a59cd9f741b1f424553cae1948dfeed661c4",
      "timestamp": "2026-01-22 12:05:49.656127"
    },
    {
      "data": "Some transaction data",
      "hash": "00007957030d4e8b2432e5037939f03d9b16d8612364ce8c6cefd2b3e2a0d5f6",
      "index": 3,
      "nonce": 30433,
      "previous_hash": "000030b5874b1e81cad31a1eb57cbcf69db993bbb0af36834c9742c4bea27ab5",
      "timestamp": "2026-01-22 12:06:04.610851"
    },
    {
      "data": "Some transaction data",
      "hash": "00002bbdd893c67f29a8d2ec5e4fe64e4c8cbc836084bb3ca6944b2a60f29e69",
      "index": 4,
      "nonce": 58439,
      "previous_hash": "00007957030d4e8b2432e5037939f03d9b16d8612364ce8c6cefd2b3e2a0d5f6",
      "timestamp": "2026-01-22 12:06:09.082474"
    }
  ],
  "length": 4
}
```



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Roll No.	54
Name	Arnav Sawant
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3: Implement smart contracts in Ethereum using different development frameworks. CO5: Interpret different Crypto assets and Crypto currencies

Blockchain Lab EXP 3

Aim - Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory -

1. Blockchain Overview

A blockchain is a **distributed, decentralized digital ledger** used to record transactions securely across multiple computers (nodes). Instead of relying on a central authority, every node in the network maintains its own copy of the blockchain.

Each block in the blockchain contains:

- A list of transactions
- A timestamp
- The hash of the previous block
- Its own cryptographic hash

The blocks are linked together using hashes, forming a chain. Any attempt to modify data in a block changes its hash, which breaks the chain and makes tampering easily detectable. This property ensures **data integrity, transparency, and immutability**.

2. Mining

Mining is the process by which new blocks are added to the blockchain. It involves:

- Collecting pending transactions
- Creating a block header
- Solving a computationally difficult problem known as **Proof-of-Work (PoW)**

In Proof-of-Work, miners repeatedly change a nonce value to generate a hash that satisfies the network's difficulty requirement (such as leading zeroes). Once a valid hash is found, the block is added to the blockchain and shared with other nodes. As an incentive, miners receive a **cryptocurrency reward** for their computational effort.

3. Multi-Node Blockchain Network

In this experiment, a **multi-node blockchain network** is simulated using three nodes running on different ports (5001, 5002, and 5003).

Each node:

- Operates independently
- Maintains its own copy of the blockchain
- Communicates with peer nodes to share newly mined blocks

This setup demonstrates how blockchain achieves decentralization and synchronization without a central server.

4. Consensus Mechanism

To ensure consistency across all nodes, the **Longest Chain Rule** is used as the consensus mechanism.

When different versions of the blockchain exist, the node accepts the **longest valid chain** as the correct one. This rule ensures that all nodes eventually agree on a single transaction history, even if temporary differences arise.

5. Transactions and Mining Reward

Transactions in the cryptocurrency system include:

- Sender address
- Receiver address
- Transaction amount

When a block is mined, all pending transactions are added to the block. Additionally, a special transaction is created to reward the miner with newly generated cryptocurrency. This reward mechanism motivates miners to maintain and secure the network.

6. Chain Replacement

The chain replacement process ensures network consistency. When the `/replace_chain` endpoint is triggered:

- A node requests blockchain data from its peers
- Validates the received chains
- Replaces its own chain if a longer and valid chain is found

This mechanism helps resolve conflicts and keeps all nodes synchronized.

Code -

```
class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
        self.create_block(proof=1, previous_hash='0')
        self.nodes = set()
    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,
```

```

        'timestamp': str(datetime.datetime.now()),
        'proof': proof,
        'previous_hash': previous_hash,
        'transactions': self.transactions
    }
    self.transactions = []
    self.chain.append(block)
    return block

def get_previous_block(self):
    return self.chain[-1]

def proof_of_work(self, previous_proof):
    new_proof = 1
    check_proof = False
    while not check_proof:
        hash_operation = hashlib.sha256(
            str(new_proof**2 - previous_proof**2).encode()
        ).hexdigest()
        if hash_operation[:4] == '0000':
            check_proof = True
        else:
            new_proof += 1
    return new_proof

def hash(self, block):
    encoded_block = json.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        if block['previous_hash'] != self.hash(previous_block):
            return False
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(
            str(proof**2 - previous_proof**2).encode()

```

```

        ).hexdigest()
        if hash_operation[:4] != '0000':
            return False
        previous_block = block
        block_index += 1
    return True

def add_transaction(self, sender, receiver, amount):
    self.transactions.append({
        'sender': sender,
        'receiver': receiver,
        'amount': amount
    })
    previous_block = self.get_previous_block()
    return previous_block['index'] + 1

def add_node(self, address):
    parsed_url = urlparse(address)
    self.nodes.add(parsed_url.netloc)

def replace_chain(self):
    network = self.nodes
    longest_chain = None
    max_length = len(self.chain)
    for node in network:
        response = requests.get(f'http://{node}/get_chain')
        if response.status_code == 200:
            length = response.json()['length']
            chain = response.json()['chain']
            if length > max_length and self.is_chain_valid(chain):
                max_length = length
                longest_chain = chain
    if longest_chain:
        self.chain = longest_chain
        return True
    return False

app = Flask(__name__)

node_address = str(uuid4()).replace('-', '')

```

```
blockchain = Blockchain()

@app.route('/mine_block', methods=['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    blockchain.add_transaction(
        sender=node_address,
        receiver='Richard',
        amount=1
    )
    block = blockchain.create_block(proof, previous_hash)
    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions']
    }
    return jsonify(response), 200

@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }
    return jsonify(response), 200

@app.route('/is_valid', methods=['GET'])
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
        response = {'message': 'The Blockchain is not valid.'}
```

```
    return jsonify(response), 200

@app.route('/add_transaction', methods=['POST'])
def add_transaction():
    json_data = request.get_json()
    transaction_keys = ['sender', 'receiver', 'amount']
    if not all(key in json_data for key in transaction_keys):
        return 'Some elements of the transaction are missing', 400
    index = blockchain.add_transaction(
        json_data['sender'],
        json_data['receiver'],
        json_data['amount']
    )
    response = {
        'message': f'This transaction will be added to Block {index}'
    }
    return jsonify(response), 201

@app.route('/connect_node', methods=['POST'])
def connect_node():
    json_data = request.get_json()
    nodes = json_data.get('nodes')
    if nodes is None:
        return "No node", 400
    for node in nodes:
        blockchain.add_node(node)
    response = {
        'message': 'All the nodes are now connected.',
        'total_nodes': list(blockchain.nodes)
    }
    return jsonify(response), 201

@app.route('/replace_chain', methods=['GET'])
def replace_chain():
    is_chain_replaced = blockchain.replace_chain()
    if is_chain_replaced:
        response = {
            'message': 'The chain was replaced by the longest one.',
            'new_chain': blockchain.chain
        }
    }
```

```

else:
    response = {
        'message': 'The chain is already the largest one.',
        'actual_chain': blockchain.chain
    }
    return jsonify(response), 200

app.run(host='0.0.0.0', port=5000)

```

My Collection / Get data

GET http://127.0.0.1:5001/mine_block

Send

Docs Params Authorization Headers (6) Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (5) Test Results (1/1) 200 OK 6 ms 472 B Save Response

JSON Preview Visualize

```

1 {
2   "block": {
3     "index": 2,
4     "previous_hash": "966ccab9af5bf8f89464b709015b40f4ec039b972c42ba68bf959966d08b3d84",
5     "proof": 533,
6     "timestamp": "2026-01-26 22:38:00.495959",
7     "transactions": [
8       {
9         "amount": 1,
10        "receiver": "Richard",
11        "sender": "f9e35ebae0064c50a6a30c13463e2a45"
12      }
13    ]
14  },
15  "message": "Congratulations, you just mined a block!"

```

Activate Windows
Go to Settings to activate Windows

My Collection / Get data

GET http://127.0.0.1:5001/get_chain

200 OK • 4 ms • 533 B

```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-01-26 22:35:03.442382",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "966ccab9af5bf8f89464b709015b40f4ec039b972c42ba68bf959966d08b3d84",
      "proof": 533,
      "timestamp": "2026-01-26 22:38:00.495959",
      "transactions": [
        {
          "amount": 1,
          "receiver": "Richard",
          "sender": "f9e35ebae0064c50a6a30c13463e2a45"
        }
      ]
    }
  ]
}
```

GET Get data

My Collection / Get data

GET http://127.0.0.1:5001/is_valid

200 OK • 4 ms • 199 B

```
{
  "message": "Blockchain is valid"
}
```

Conclusion - Learned to create cryptocurrency and mining process using python.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 04

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Roll No.	54
Name	Arnav Sawant
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Experiment 4

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was

successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- **Function Visibility** defines who can access a function:

- o public: available both inside and outside the contract.
- o private: only accessible within the same contract.
- o internal: accessible within the contract and its child contracts.
- o external: can be called only by external accounts or other contract

- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).

- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays:** Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings:** Key-value pairs that allow quick lookups. Example: mapping(address

=> uint) for storing balances. Unlike arrays, mappings do not support iteration.

- **Structs:** Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums:** Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

- **storage:** Data stored permanently on the blockchain. Examples: state variables.
- **memory:** Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata:** A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei:** Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit ($1 \text{ Ether} = 10^{18} \text{ Wei}$). This ensures high precision in financial transactions.
- **Gas and Gas Price:** Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions:** Transactions are used for transferring Ether or interacting with contracts. Functions like transfer() and send() are

commonly used, while `call()` provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

Implementation:-

The screenshot shows the Remix IDE interface. On the left, the 'LEARNETH' sidebar displays the '1. Introduction' tutorial. The main editor shows a Solidity contract named 'Counter' with the following code:

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) {
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public {
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public {
19        count -= 1;
20    }
21 } // Arnav Sawant D20A 54

```

The bottom status bar shows the 'AI copilot' is enabled and a 'Debug' button is visible.

The screenshot shows the Remix IDE interface. On the left, the 'LEARNETH' sidebar displays the '2. Basic Syntax' tutorial. The main editor shows a Solidity contract named 'MyContract' with the following code:

```

1 // SPDX-License-Identifier: MIT
2 // compiler version must be greater than or equal to 0.8.3 and less than 0.9.0
3 pragma solidity ^0.8.3;
4
5 contract MyContract {
6     string public name = "Alice";
7 } // Arnav Sawant D20A 54

```

The bottom status bar shows the 'AI copilot' is enabled and a 'Debug' button is visible.

LEARNETH 1.5.1

learneth tutorials

Login with GitHub Theme

3. Primitive Data Types 3 / 19

Assignment

1. Create a new variable `newAddr` that is a `public` `address` and give it a value that is not the same as the available variable `addr`.
2. Create a `public` variable called `neg` that is a negative number, decide upon the type.
3. Create a new variable, `newU` that has the smallest `uint` size type and the smallest `uint` value and is `public`.

Tip: Look at the other address in the contract or search the internet for an Ethereum address.

Check Answer

Show answer

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Primitives {
5     bool public boo = true;
6     // Arnav Sawant D20A 54 -----> Lab exp4
7     /*
8      uint stands for unsigned integer, meaning non negative integers
9      different sizes are available
10     uint8   ranges from 0 to 2 ** 8 - 1
11     uint16  ranges from 0 to 2 ** 16 - 1
12     ...
13     uint256 ranges from 0 to 2 ** 256 - 1
14     */
15     uint8 public u8 = 1;
16     uint public u256 = 456;
17     uint public u = 123; // uint is an alias for uint256
18 }
19

```

Explain contract

- Check transactions details and start debugging.
- Execute JavaScript scripts:
 - Input a script directly in the command line interface
 - Select a JavaScript file in the file explorer and then run `remix.execute()` or `remix.executeCurrent()` in the command line interface
 - Right-click on a JavaScript file in the file explorer and then click "Run"

The following libraries are accessible:

- `ethers.js`

Type the library name to see available commands.

LEARNETH 1.5.1

learneth tutorials

Login with GitHub Theme

4. Variables 4 / 19

the Solidity documentation.

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer

Show answer

Next

Well done! No errors.

```

2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Hello";
7     uint public num = 123;
8     uint public blockNumber;
9
10    function doSomething() public {
11        // Local variables are not saved to the blockchain.
12        uint i = 456;
13        // Arnav Sawant D20A 54 ----->
14        // Here are some global variables
15        uint timestamp = block.timestamp; // Current block timestamp
16        address sender = msg.sender; // address of the caller
17        blockNumber = block.number;
18    }
19 }

```

Explain contract

- Check transactions details and start debugging.
- Execute JavaScript scripts:
 - Input a script directly in the command line interface
 - Select a JavaScript file in the file explorer and then run `remix.execute()` or `remix.executeCurrent()` in the command line interface
 - Right-click on a JavaScript file in the file explorer and then click "Run"

The following libraries are accessible:

- `ethers.js`

Type the library name to see available commands.

LEARNETH 1.5.1 learneth tutorials Login with GitHub Theme

Tutorials list **Syllabus**

5.1 Functions - Reading and Writing to a State Variable 5/19

types in this case, `uint` (unsigned integer) since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video tutorial on Functions.

Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

Check Answer

Show answer

Next

Well done! No errors.

```

5 // state variable to store a number
6 uint public num;
7 bool public b = true;
8
9 // You need to send a transaction to write to a state variable.
10 function set(uint _num) public { 22536 gas
11     num = _num;
12 }
13
14 // You can read from a state variable without sending a transaction.
15 function get() public view returns (uint) { 2475 gas
16     return num;
17 }
18
19 function get_b() public view returns (bool) { 2539 gas
20     return b;
21 }
22

```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

- Check transactions details and start debugging.
- Execute JavaScript scripts:
 - Input a script directly in the command line interface
 - Select a JavaScript file in the file explorer and then run `remix.execute()` or `remix.executeCurrent()` in the command line interface
 - Right-click on a JavaScript file in the file explorer and then click "Run"

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

LEARNETH 1.5.1 learneth tutorials Login with GitHub Theme

Tutorials list **Syllabus**

5.2 Functions - View and Pure 6/19

computation cost (gas cost). Declaring functions `view` and `pure` can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

Watch a video tutorial on View and Pure Functions.

Assignment

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer

Show answer

Next

Well done! No errors.

```

4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addToX(uint y) public view returns (uint) { infinite gas
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) { infinite gas
14        return i + j;
15    }
16
17    function addToX2(uint y) public { infinite gas
18        x = x + y;
19    }
20 }

```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

- Check transactions details and start debugging.
- Execute JavaScript scripts:
 - Input a script directly in the command line interface
 - Select a JavaScript file in the file explorer and then run `remix.execute()` or `remix.executeCurrent()` in the command line interface
 - Right-click on a JavaScript file in the file explorer and then click "Run"

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

The screenshot shows the Remix IDE interface. On the left, the 'LEARNETH' sidebar displays the 'Tutorials list' and 'Syllabus' for '5.3 Functions - Modifiers and Constructors'. The 'Assignment' section lists three tasks: 1. Create a new function, 'increaseX', in the contract. The function should take an input parameter of type 'uint' and increase the value of the variable 'x' by the value of the input parameter. 2. Make sure that x can only be increased. 3. The body of the function 'increaseX' should be empty. Below the tasks are buttons for 'Check Answer', 'Show answer', 'Next', and a green bar indicating 'Well done! No errors.'.

The main editor shows a Solidity file named 'modifiersAndConstructors.sol'. The code includes a 'noReentrancy' modifier and a 'decrement' function. A comment indicates the file is for 'Arnav Sawant D20A 54'. The bottom panel shows the 'Explain contract' section with instructions on how to check transactions and execute JavaScript scripts.

```

51 // This modifier prevents a function from being called while
52 // it is still executing.
53 modifier noReentrancy() {
54     require(!locked, "No reentrancy");
55
56     locked = true;
57     _;
58     locked = false;
59 }
60
61 function decrement(uint i) public noReentrancy {
62     x -= i;
63 }
64 // Arnav Sawant D20A 54 ----->
65 if (i > 1) {
66     decrement(i - 1);
67 }
68 }

```

The screenshot shows the Remix IDE interface. On the left, the 'LEARNETH' sidebar displays the 'Tutorials list' and 'Syllabus' for '5.4 Functions - Inputs and Outputs'. The 'Assignment' section lists one task: Create a new function called 'returnTwo' that returns the values '-2' and 'true' without using a return statement. Below the task are buttons for 'Check Answer', 'Show answer', 'Next', and a green bar indicating 'Well done! No errors.'.

The main editor shows a Solidity file named 'inputsAndOutputs.sol'. The code includes an 'arrayOutput' function and a 'returnTwo' function. A comment indicates the file is for 'Arnav Sawant D20A 54'. The bottom panel shows the 'Explain contract' section with instructions on how to check transactions and execute JavaScript scripts.

```

74 uint[] public arr;
75
76 function arrayOutput() public view returns (uint[] memory) {
77     return arr;
78 }
79
80 function returnTwo() {
81     public
82     pure
83     returns (
84         int i,
85         bool b
86     )
87     {
88         i = -2;
89         b = true;
90     }
91 } // Arnav Sawant D20A 54 ----->

```

The screenshot shows the REMIX IDE interface. On the left, the 'LEARNETH' sidebar displays a tutorial for '6. Visibility' (9/19). The main editor shows a Solidity contract named 'Child' that inherits from 'Base'. The contract defines two functions: 'testPrivateFunc' and 'testInternalFunc'. The 'testPrivateFunc' function is marked as 'private' and returns a string. The 'testInternalFunc' function is marked as 'internal' and returns a string. The contract also defines a 'testInternalVar' function that returns a string. The 'Child' contract is shown as follows:

```

53 }
54
55 contract Child is Base {
56     // Inherited contracts do not have access to private functions
57     // and state variables.
58     function testPrivateFunc() public pure returns (string memory) {
59         return privateFunc();
60     }
61
62     // Internal function call be called inside child contracts.
63     function testInternalFunc() public pure override returns (string memory) {
64         return internalFunc();
65     }
66
67     function testInternalVar() public view returns (string memory, string memory) {
68         return (internalVar, publicVar);
69     }
70 }

```

Below the code editor, the 'Explain contract' section provides details about the contract and its functions. It includes a list of instructions for checking transactions and debugging, and a section for the accessible libraries.

Check transactions details and start debugging.

- Execute JavaScript scripts:
 - Input a script directly in the command line interface
 - Select a JavaScript file in the file explorer and then run 'remix.execute()' or 'remix.executeCurrent()' in the command line interface
 - Right-click on a JavaScript file in the file explorer and then click 'Run'

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

At the bottom, there are buttons for 'Check Answer', 'Show answer', and 'Next'. A green message at the bottom says 'Well done! No errors.'

The screenshot shows the REMIX IDE interface. On the left, the 'LEARNETH' sidebar displays a tutorial for '7.1 Control Flow - If/Else' (10/19). The main editor shows a Solidity contract named 'IIfElse'. The contract defines two functions: 'foo' and 'ternary'. The 'foo' function is marked as 'public' and returns a 'uint'. The 'ternary' function is marked as 'public' and returns a 'uint'. The 'foo' function is shown as follows:

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 // Arnav Sawant D20A 54
4 contract IIfElse {
5     function foo(uint x) public pure returns (uint) {
6         if (x < 10) {
7             return 0;
8         } else if (x < 20) {
9             return 1;
10        } else {
11            return 2;
12        }
13    }
14
15    function ternary(uint _x) public pure returns (uint) {
16        // if (_x < 10) {
17        //     return 1;
18        // }
19        // return 2;
20
21        // shorthand way to write if / else statement
22        return _x < 10 ? 1 : 2;
23    }
24
25    function evenCheck(uint v) public pure returns (bool) {
26
27    }
28 }

```

Below the code editor, the 'Explain contract' section provides details about the contract and its functions. It includes a list of instructions for checking transactions and debugging, and a section for the accessible libraries.

Check transactions details and start debugging.

- Execute JavaScript scripts:
 - Input a script directly in the command line interface
 - Select a JavaScript file in the file explorer and then run 'remix.execute()' or 'remix.executeCurrent()' in the command line interface
 - Right-click on a JavaScript file in the file explorer and then click 'Run'

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

At the bottom, there are buttons for 'Check Answer', 'Show answer', and 'Next'. A green message at the bottom says 'Well done! No errors.'

LEARNETH 1.5.1

learneth tutorials

Login with GitHub Theme

7.2 Control Flow - Loops 11 / 19

executed at least once, before checking on the condition.

continue

The `continue` statement is used to skip the remaining code block and start the next iteration of the loop. In this contract, the `continue` statement (line 10) will prevent the second if statement (line 12) from being executed.

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

★ **Assignment**

1. Create a public `uint` state variable called count in the `Loop` contract.
2. At the end of the for loop, increment the count variable by 1.
3. Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer Show answer

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 // Arnav Sawant D20A 54
4 contract Loop {
5     uint public count;
6     function loop() public {
7         // for loop
8         for (uint i = 0; i < 10; i++) {
9             if (i == 5) {
10                // Skip to next iteration with continue
11                continue;
12            }
13            if (i == 5) {
14                // Exit loop with break
15                break;
16            }
17            count++;
18        }
19    }
20    // while loop
21    uint j;
22    while (j < 10) {
23        j++;
24    }
25 }

```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

Type the library name to see available commands.

LEARNETH 1.5.1

learneth tutorials

Login with

8.1 Data Structures - Arrays 12 / 19

our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

★ **Assignment**

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer Show answer

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 // Arnav Sawant D20A 54
4 contract Array {
5     // Several ways to initialize an array
6     uint[] public arr;
7     uint[] public arr2 = [1, 2, 3];
8     // Fixed sized array, all elements initialize to 0
9     uint[10] public myFixedSizeArr;
10    uint[3] public arr3 = [0, 1, 2];
11
12    function get(uint i) public view returns (uint) {
13        return arr[i];
14    }
15
16    // Solidity can return the entire array.
17    // But this function should be avoided for
18    // arrays that can grow indefinitely in length.
19    function getArr() public view returns (uint[3] memory) {
20        return arr3;
21    }
22
23    function push(uint i) public {
24        // Append to array
25        // This will increase the array length by 1.

```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

Type the library name to see available commands.

REMIX 1.5.1

learneth tutorials

LEARNETH

Tutorials list

8.2 Data Structures - Mappings

we can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section. Watch a video tutorial on Mappings.

★ **Assignment**

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping balances.
3. Change the function `set` to create a new entry to the balances mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer Show answer

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 // Arnav Sawant D20A 54
4 contract Mapping {
5     // Mapping from address to uint
6     mapping(address => uint) public balances;
7
8     function get(address _addr) public view returns (uint) {
9         // Mapping always returns a value.
10        // If the value was never set, it will return the default value.
11        return balances[_addr];
12    }
13
14    function set(address _addr) public {
15        // Update the value at this address
16        balances[_addr] = _addr.balance;
17    }
18
19    function remove(address _addr) public {
20        // Reset the value to the default value.
21        delete balances[_addr];
22    }
23 }
24
25 contract NestedMapping {

```

Explain contract

0 Listen on all transactions Filter with transaction

Type the library name to see available commands.

REMIX 1.5.1

learneth tutorials

LEARNETH

Tutorials list

8.3 Data Structures - Structs

Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a structs' member we also use the dot operator and assign it a new value (lines 39 and 45). Watch a video tutorial on Structs.

★ **Assignment**

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer Show answer

Next

Well done! No errors.

```

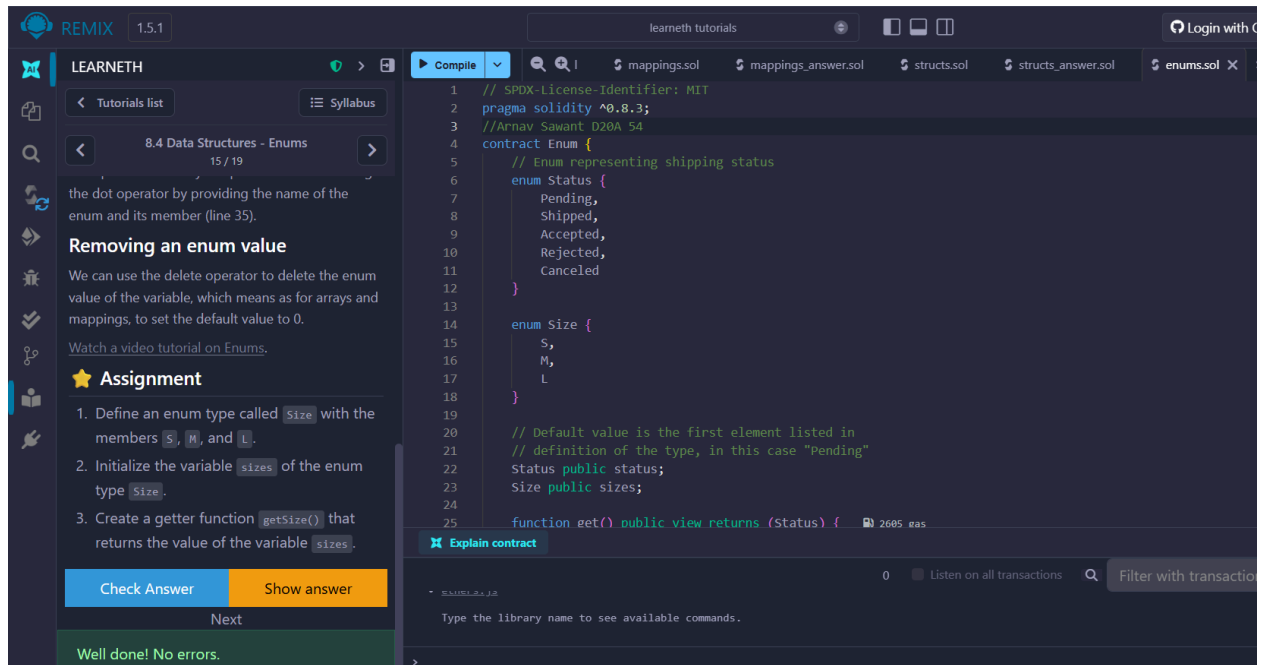
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 // Arnav Sawant D20A 54
4 contract Todos {
5     struct Todo {
6         string text;
7         bool completed;
8     }
9
10    // An array of 'Todo' structs
11    Todo[] public todos;
12
13    function create(string memory _text) public {
14        // 3 ways to initialize a struct
15        // - calling it like a function
16        todos.push(Todo(_text, false));
17
18        // key value mapping
19        todos.push(Todo({text: _text, completed: false}));
20
21        // initialize an empty struct and then update it
22        Todo memory todo;
23        todo.text = _text;
24        // todo.completed initialized to false
25    }

```

Explain contract

0 Listen on all transactions Filter with transaction

Type the library name to see available commands.



LEARNETH

Tutorials list

Syllabus

8.4 Data Structures - Enums

15 / 19

the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer Show answer

Next

Well done! No errors.

```

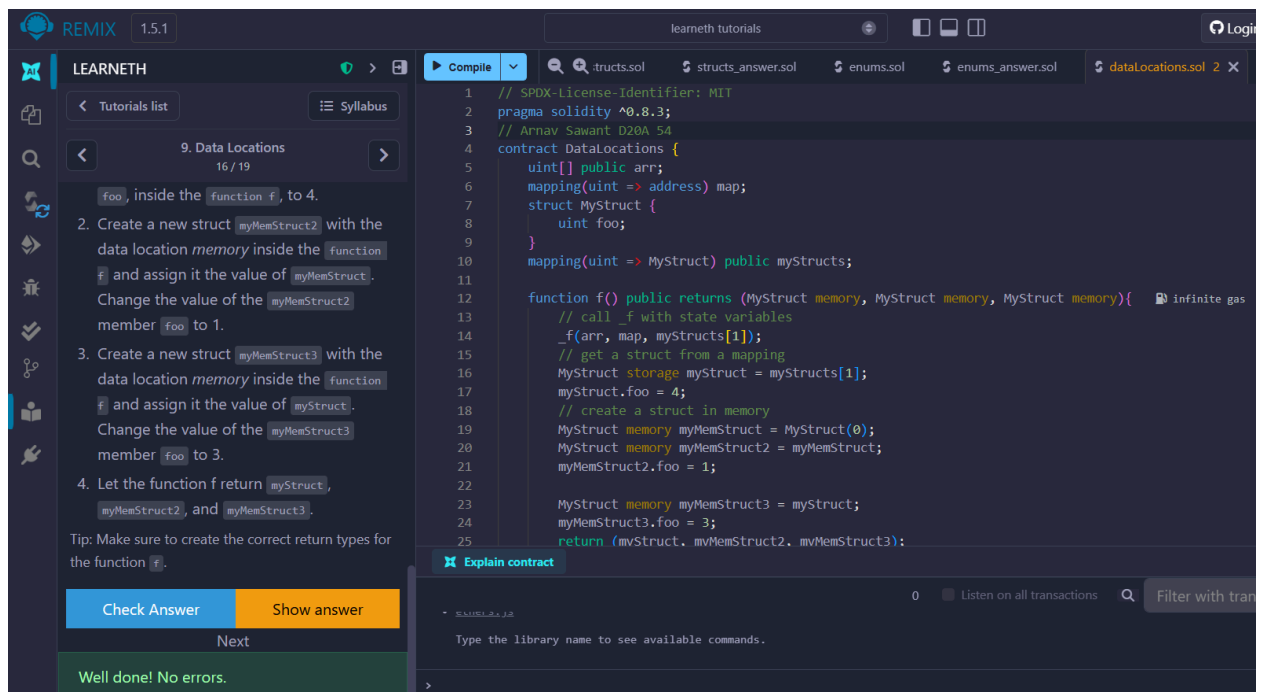
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Arnav Sawant D20A 54
4 contract Enum {
5     // Enum representing shipping status
6     enum Status {
7         Pending,
8         Shipped,
9         Accepted,
10        Rejected,
11        Canceled
12    }
13
14    enum Size {
15        S,
16        M,
17        L
18    }
19
20    // Default value is the first element listed in
21    // definition of the type, in this case "Pending"
22    Status public status;
23    Size public sizes;
24
25    function get() public view returns (Status) {
26
27    }
28 }

```

Explain contract

0 Listen on all transactions Filter with transaction

Type the library name to see available commands.



LEARNETH

Tutorials list

Syllabus

9. Data Locations

16 / 19

`foo`, inside the `function f`, to 4.

2. Create a new struct `myMemStruct2` with the data location `memory` inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer Show answer

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 // Arnav Sawant D20A 54
4 contract DataLocations {
5     uint[] public arr;
6     mapping(uint => address) map;
7     struct MyStruct {
8         uint foo;
9     }
10    mapping(uint => MyStruct) public myStructs;
11
12    function f() public returns (MyStruct memory, MyStruct memory, MyStruct memory){
13        // call f with state variables
14        _f(arr, map, myStructs[1]);
15        // get a struct from a mapping
16        MyStruct storage myStruct = myStructs[1];
17        myStruct.foo = 4;
18        // create a struct in memory
19        MyStruct memory myMemStruct = MyStruct(0);
20        MyStruct memory myMemStruct2 = myMemStruct;
21        myMemStruct2.foo = 1;
22
23        MyStruct memory myMemStruct3 = myStruct;
24        myMemStruct3.foo = 3;
25        return (myStruct, myMemStruct2, myMemStruct3);
26    }
27 }

```

Explain contract

0 Listen on all transactions Filter with transaction

Type the library name to see available commands.

REMUX 1.5.1

LEARNETH

Tutorials list | Syllabus

10.1 Transactions - Ether and Wei (17 / 19)

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

One `ether` is equal to 1,000,000,000,000,000,000 (10^{18}) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer **Show answer**

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12
13    uint public oneGwei = 1 gwei;
14    // 1 ether is equal to 10^9 wei
15    bool public isOneGwei = 1 gwei == 1e9;
16 } // Arnav Sawant D20A 54
    
```

Explain contract

0 Listen on all transactions Filter with transaction

Type the library name to see available commands.

REMUX 1.5.1

LEARNETH

Tutorials list | Syllabus

10.2 Transactions - Gas and Gas Price (18 / 19)

completed, reverting any changes being made. In this case, the `gas` was consumed and can't be refunded.

Learn more about `gas` on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin `Gas Profiler` to check for the gas cost of transactions.

Check Answer **Show answer**

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public {
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 } // Arnav Sawant D20A 54
    
```

Explain contract

0 Listen on all transactions Filter with transaction

Type the library name to see available commands.

The screenshot shows the Remix IDE interface. On the left, the 'LEARNETH' sidebar lists tutorials, with '10.3 Transactions - Sending Ether' (19/19) selected. The main editor displays the following Solidity code:

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 // Arnav Sawant D20A 54
4 contract ReceiveEther {
5     // Function to receive Ether. msg.data must be empty
6     receive() external payable {}
7
8     // Fallback function is called when msg.data is not empty
9     fallback() external payable {}
10
11     function getBalance() public view returns (uint) {
12         return address(this).balance;
13     }
14 }
15
16 contract SendEther {
17     function sendViaTransfer(address payable _to) public payable {
18         // This function is no longer recommended for sending Ether.
19         _to.transfer(msg.value);
20     }
21
22     function sendViaSend(address payable _to) public payable {
23         // Send returns a boolean value indicating success or failure.
24         // This function is not recommended for sending Ether.
25         bool sent = _to.send(msg.value);
26     }
27 }

```

Below the code editor, the 'Explain contract' section is visible, showing a search bar and a list of available commands.

Conclusion:-

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 05

Aim: Deploying a Voting/Ballot Smart Contract

Roll No.	54
Name	Arnav Sawant
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks.

EXPERIMENT 5

AIM: Deploying a Voting/Ballot Smart Contract

Tasks:

1. Open [Remix IDE](#)
2. Under **Workspaces**, open **contracts** folder
3. Open **Ballot.sol**, contract.
4. Understand **Ballot.sol** contract.
5. Deploy the contract by changing the Proposal name from **bytes32** → **string**

THEORY:

1. Relevance of require Statements in Solidity Programs

In Solidity, the require statement acts as a guard condition within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a Voting Smart Contract, require can be used to check:

- Whether the person calling the function has the right to vote
(`require(voters[msg.sender].weight > 0, "Has no right to vote");`).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the chairperson before granting voting rights.

Thus, require statements enforce security, correctness, and reliability in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier for users.

2. Keywords: mapping, storage, and memory

mapping:

- A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is `mapping(keyType => valueType)`. For example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like Ballot, where you need to associate voters with their data (whether they voted, which proposal they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them gas efficient for lookups but limited for enumeration.

storage:

- In Solidity, storage refers to the permanent memory of the contract, stored on the Ethereum blockchain. Variables declared at the contract level are stored in storage by default. Data stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However, because writing to blockchain storage consumes gas, it is more expensive. For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle.

memory:

- In contrast, memory is temporary storage, used only for the lifetime of a function call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used.

Thus, a smart contract developer must balance between storage and memory to ensure efficiency and cost-effectiveness.

- **bytes32** is a fixed-size type, meaning it always stores exactly 32 bytes of data. This makes storage simple, comparison operations faster, and gas costs lower. However, it limits proposal names to 32 characters, which is not very flexible for user-friendly names.
- **string** is a dynamically sized type, meaning it can store text of variable length. While it is easier for developers and users (since names can be written normally), it requires more complex handling inside the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or manipulations.

To make the system more user-friendly, modern implementations of the Ballot contract often convert from bytes32 to string. Tools like the Web3 Type Converter help developers easily switch between these two types for deployment and testing.

CODE:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {

    struct Voter
    { uint
      weight;
      bool voted;
      address delegate;
      uint vote;
    }

    struct Proposal {
        string name;    // CHANGED from bytes32 → string
        uint voteCount;
    }

    address public chairperson;
    mapping(address => Voter) public voters;
    Proposal[] public proposals;

    /**
     * @dev Create a new ballot
     * @param proposalNames names of proposals
     */
    constructor(string[] memory proposalNames)
    { chairperson = msg.sender;
      voters[chairperson].weight = 1;
    }
}
```

```

for (uint i = 0; i < proposalNames.length; i++)
    { proposals.push(
        Proposal({
            name: proposalNames[i],
            voteCount: 0
        })
    );
}
}

```

```

function giveRightToVote(address voter) external {
    require(msg.sender == chairperson, "Only chairperson can give right to vote");
    require(!voters[voter].voted, "The voter already voted");
    require(voters[voter].weight == 0, "Voter already has right");

    voters[voter].weight = 1;
}

```

```

function delegate(address to) external
{
    Voter storage sender =
    voters[msg.sender];

    require(sender.weight != 0, "No right to vote");
    require(!sender.voted, "Already voted");
    require(to != msg.sender, "Self-delegation not allowed");

    while (voters[to].delegate != address(0))
    {
        to = voters[to].delegate;
        require(to != msg.sender, "Delegation loop detected");
    }
}

```

```

Voter storage delegate_ = voters[to];
require(delegate_.weight >= 1, "Delegate has no right");

sender.voted = true;

```

```

sender.delegate = to;

if (delegate_.voted)
    { proposals[delegate_.vote].voteCount +=
      sender.weight;
    } else {
      delegate_.weight += sender.weight;
    }
}

function vote(uint proposal) external {
    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "No right to vote");
    require(!sender.voted, "Already voted");

    sender.voted = true;
    sender.vote = proposal;

    proposals[proposal].voteCount += sender.weight;
}

function winningProposal() public view returns (uint winningProposal_)
    { uint winningVoteCount = 0;

    for (uint p = 0; p < proposals.length; p++) {
        if (proposals[p].voteCount > winningVoteCount)
            { winningVoteCount = proposals[p].voteCount;
              winningProposal_ = p;
            }
        }
    }

function winnerName() external view returns (string memory)
    { return proposals[winningProposal()].name;

```

```
}  
}
```

OUTPUT:

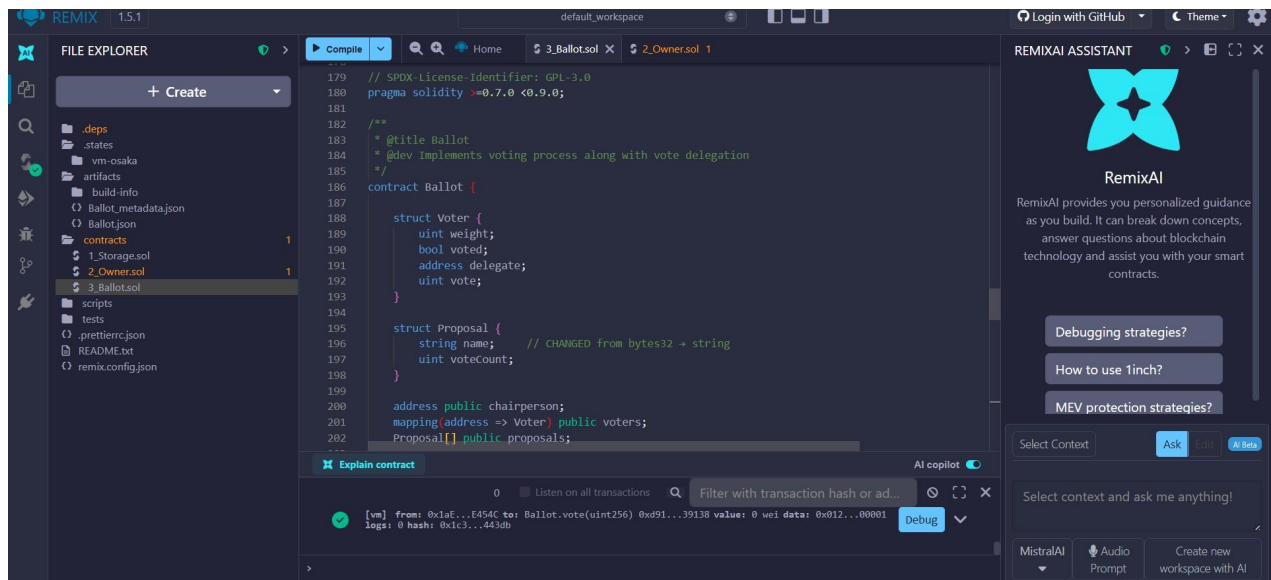
Step 1: Open Remix

Go to <https://remix.ethereum.org>

No install needed.

Step 2: Create the file

- In File Explorer
Ballot.sol



Step 3: Compile the contract

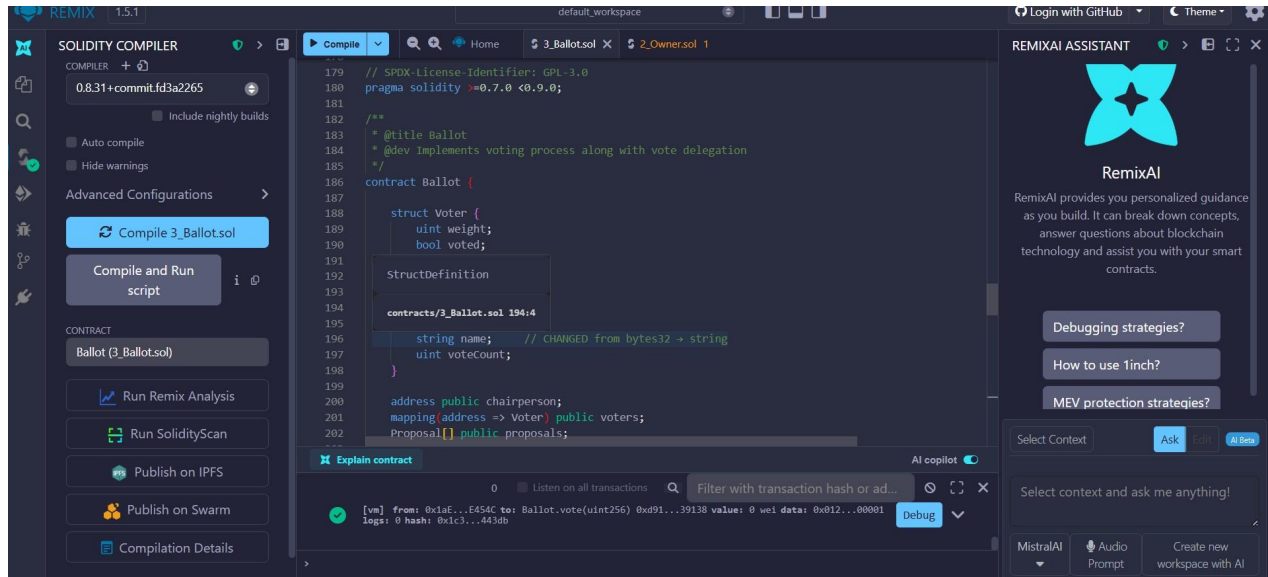
1. Open Solidity Compiler (left sidebar)
2. Click Compile Ballot.sol
3. Make sure green tick appears (no errors)

Step 4: Deploy the contract

1. Open Deploy & Run Transactions

2. Set:

- Environment → **Remix VM**
- Account → keep default
- Contract → **Ballot**



Step 5: Enter constructor input (IMPORTANT)

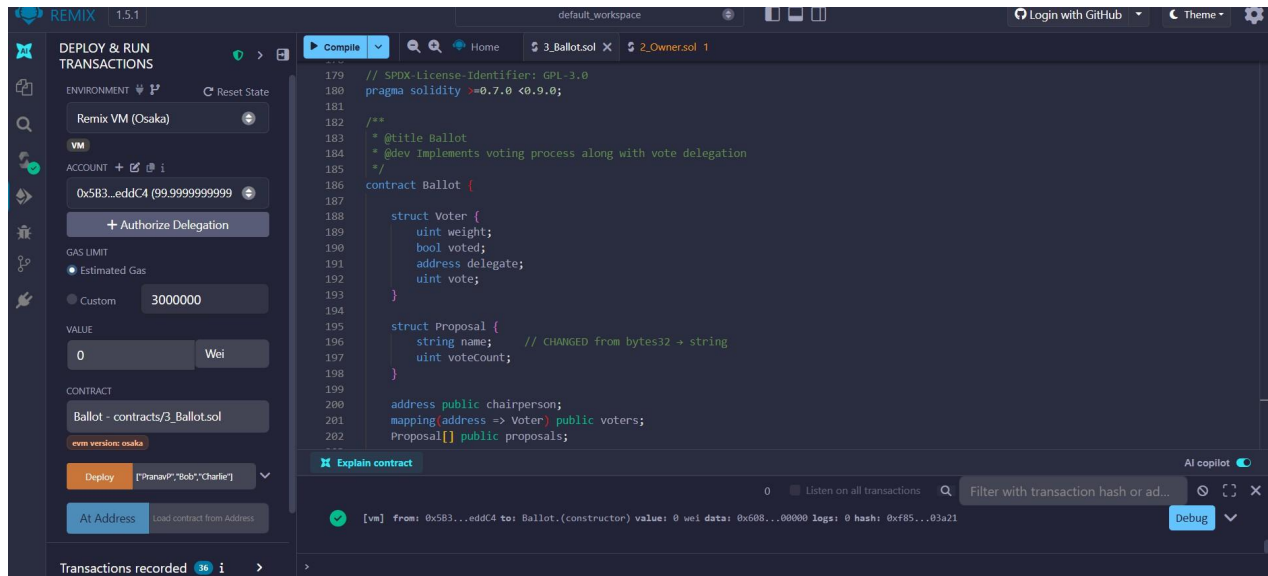
Your constructor is:

constructor(string[] memory proposalNames)

So input must be a string array

Example:

["Arnav", "Bob", "Charlie"]



Step 6: Click Deploy

- Click Deploy
- Contract appears under Deployed Contracts
- The deployer is automatically the chairperson

giveRightToVote (address voter)

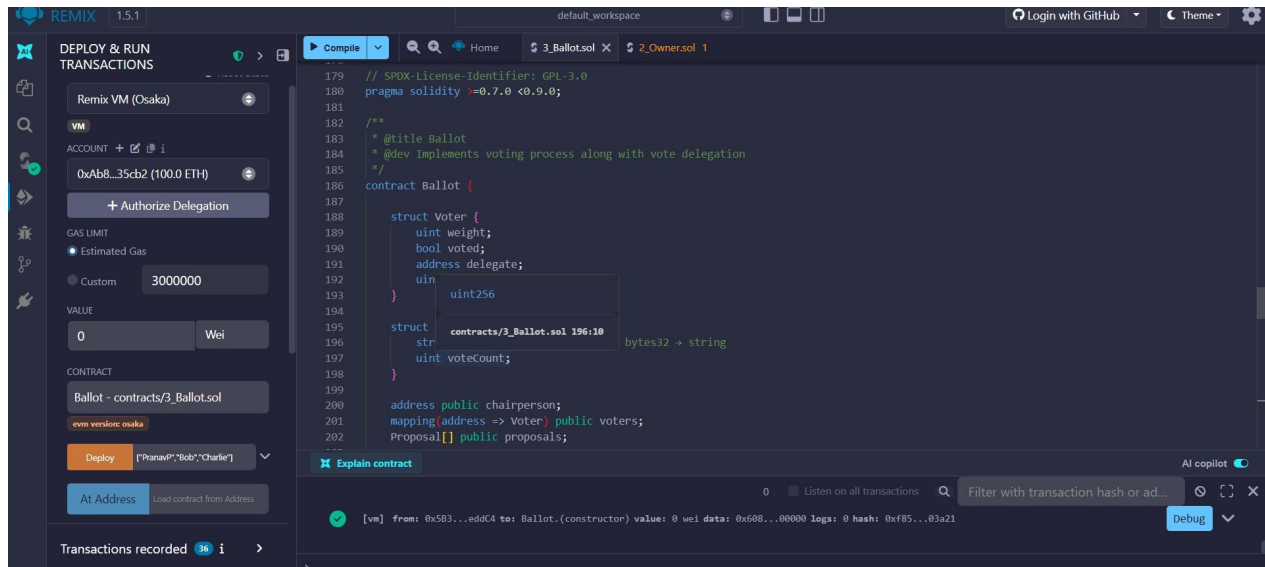
What to add:

Paste a Remix account address (NOT the chairperson).

Important:

- You must be using Account 1 (chairperson) when clicking this
- Switch account at the top if needed

Click Transact



Then vote

2 vote (uint256 proposal)

What to add:

A number, based on proposal index:

0 → Arnav

1 → Bob

2 → Charlie

Example:

0

Important:

- Switch to the voter account (the one you gave rights to)
- Then click Transact

REMIX 1.5.1 default_workspace Login with GitHub Theme

DEPLOY & RUN TRANSACTIONS

Deployed Contracts 1

BALLOT AT 0XD91...39138 (ME)

Balance: 0 ETH

delegate address to

giveRightToVote 0xb0483f649c6d11cf9b

vote uint256 proposal

chairperson

proposals uint256

voters address

winnerName

winningProposal

Low level interactions

CALLDATA

Transact

```
179 // SPDX-License-Identifier: GPL-3.0
180 pragma solidity >=0.7.0 <0.9.0;
181
182 /**
183  * @title Ballot
184  * @dev Implements voting process along with vote delegation
185  */
186 contract Ballot {
187
188     struct Voter {
189         uint weight;
190         bool voted;
191         address delegate;
192         uint vote;
193     }
194
195     struct Proposal {
196         string name; // CHANGED from bytes32 to string
197         uint voteCount;
198     }
199
200     address public chairperson;
201     mapping(address => Voter) public voters;
202     Proposal[] public proposals;
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or ad...

[vm] from: 0x583...edd4 to: Ballot.(constructor) value: 0 wei data: 0x688...0000 logs: 0 hash: 0xf85...83a21

Debug

REMIX 1.5.1 default_workspace Login with GitHub Theme

DEPLOY & RUN TRANSACTIONS

Transactions recorded 10

Deployed Contracts 1

BALLOT AT 0XD91...39138 (ME)

Balance: 0 ETH

delegate address to

giveRightToVote 0xb0483f649c6d11cf9b

vote 0

chairperson

proposals uint256

voters address

winnerName

winningProposal

Low level interactions

CALLDATA

Transact

```
179 // SPDX-License-Identifier: GPL-3.0
180 pragma solidity >=0.7.0 <0.9.0;
181
182 /**
183  * @title Ballot
184  * @dev Implements voting process along with vote delegation
185  */
186 contract Ballot {
187
188     struct Voter {
189         uint weight;
190         bool voted;
191         address delegate;
192         uint vote;
193     }
194
195     struct Proposal {
196         string name; // CHANGED from bytes32 to string
197         uint voteCount;
198     }
199
200     address public chairperson;
201     mapping(address => Voter) public voters;
202     Proposal[] public proposals;
```

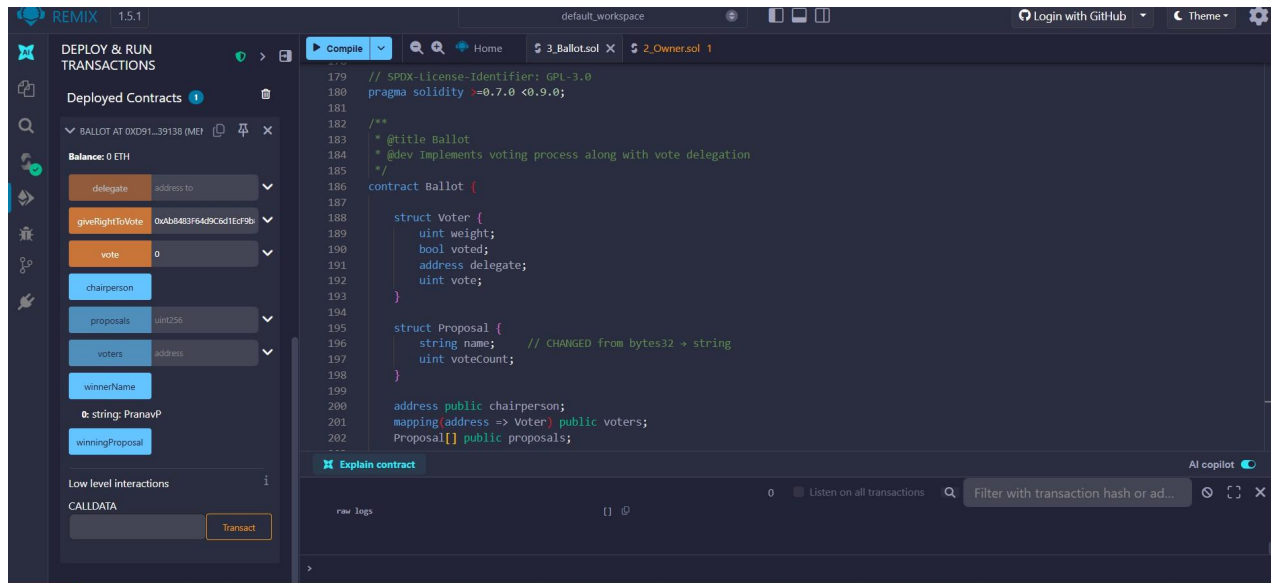
Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or ad...

[vm] from: 0x4b8...35cb2 to: Ballot.vote(uint256) 0xd91...39138 value: 0 wei data: 0x012...0000 logs: 0 hash: 0xc66...d2510

Debug



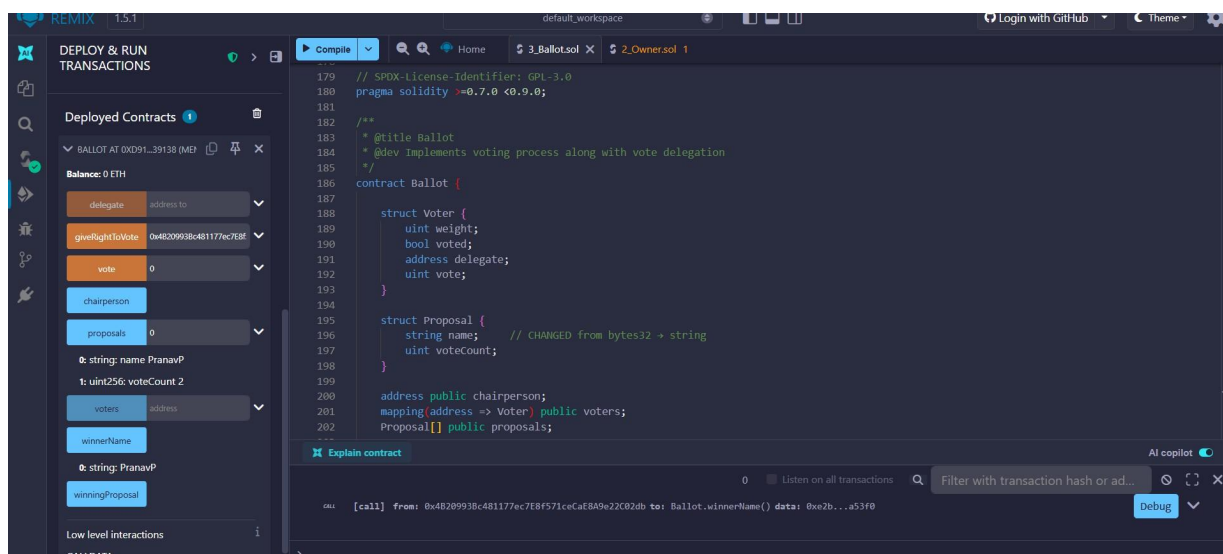
proposals (uint256)

What to add:

Proposal index:

0

Returns proposal details.



Conclusion:

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 06

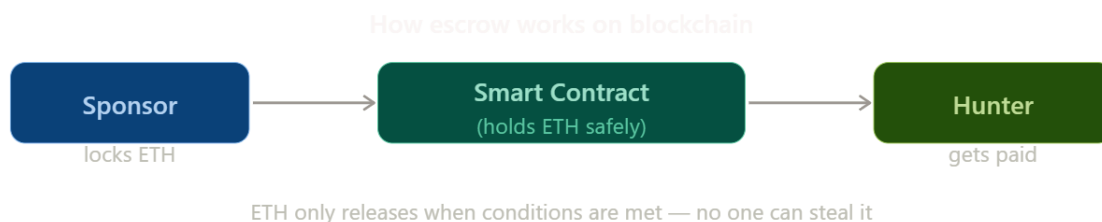
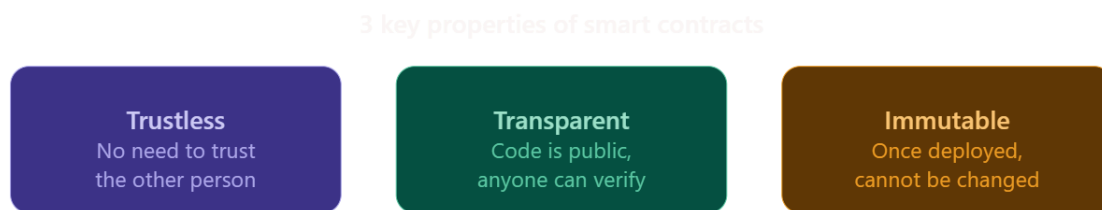
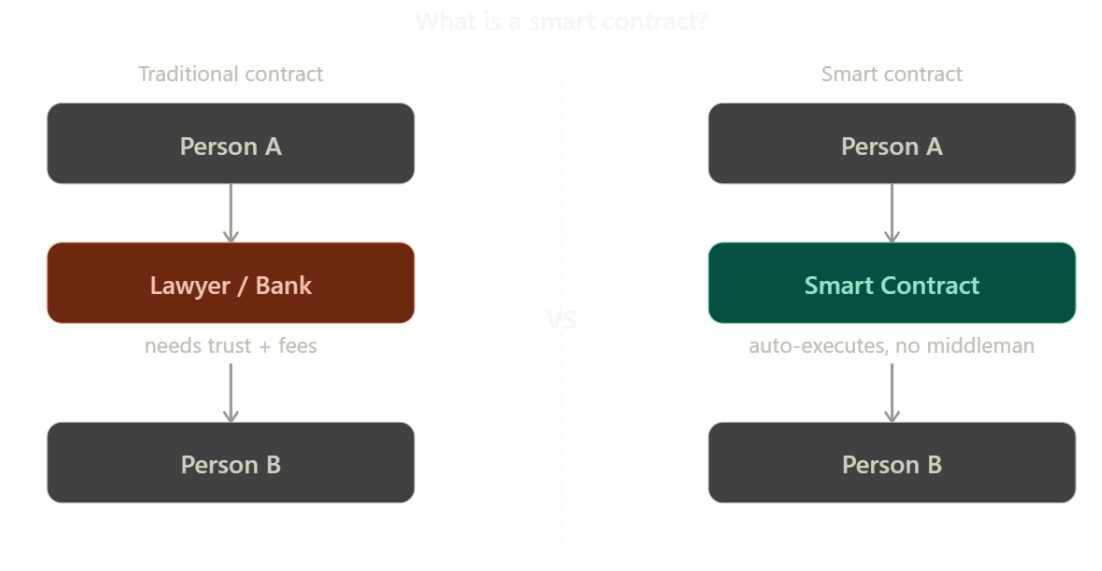
Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Roll No.	54
Name	Arnav Sawant
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Experiment 6

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:- Smart contracts are self-executing programs that live on a blockchain. Think of them like a vending machine — you put in money, the machine checks the rules, and automatically gives you what you paid for. No human in the middle.



Decentralized Bug Bounty Escrow System

What is this project?

This is a blockchain-based platform that connects **companies (sponsors)** who want their software tested, with **security researchers (hunters)** who find vulnerabilities — all without trusting a middleman.

When a sponsor posts a bounty and locks ETH, the money is held securely by a smart contract on Ethereum. The hunter only gets paid when their bug report is approved. If there's a disagreement, a neutral arbiter resolves the dispute. Everything is automated, transparent, and tamper-proof.

The Problem it Solves

Traditional bug bounty platforms like HackerOne or Bugcrowd charge high fees, can delay payments for months, and hunters have to trust the company will actually pay. With this system, the ETH is locked upfront — the hunter knows the money is there before they even start working.

The 3 Smart Contracts

BugBountyToken (BBT) — A reward token given to hunters every time they get paid. Acts like a reputation badge — the more bugs you find, the more BBT you earn.

BugBountyEscrow — The heart of the system. It locks ETH, manages the full bounty lifecycle (create → fund → submit → approve/dispute), and automatically pays the hunter when approved.

BugBountyRegistry — A public leaderboard and directory. Stores bounty listings, severity tags, and tracks each hunter's reputation score on-chain for everyone to see.

Who uses it?

Role	What they do
 Sponsor	Posts a bounty, locks ETH, reviews reports
 Hunter	Finds bugs, submits reports, earns ETH + BBT tokens
 Arbiter	Resolves disputes fairly between sponsor and hunter
 Platform Owner	Collects a small fee, manages the system

Tech Stack

Built with **Solidity 0.8.19** on the **Ethereum** blockchain. Deployable on any EVM-compatible network (Ethereum, Polygon, Base, etc.) and fully testable on **Remix IDE** with no real money needed.

Implementation:-

Step 1:- Deploy Contract 1: BugBountyToken

Step 2:- Deploy Contract 2: BugBountyEscrow

In the **Contract** dropdown, select BugBountyEscrow

Fill in the constructor arguments:

- `_rewardToken`: paste the BugBountyToken address from Step 3
- `_platformFeeBps`: enter 200 (= 2% platform fee)

Click **Deploy**

Copy the BugBountyEscrow deployed address

Step 3:- Deploy Contract 3: BugBountyRegistry

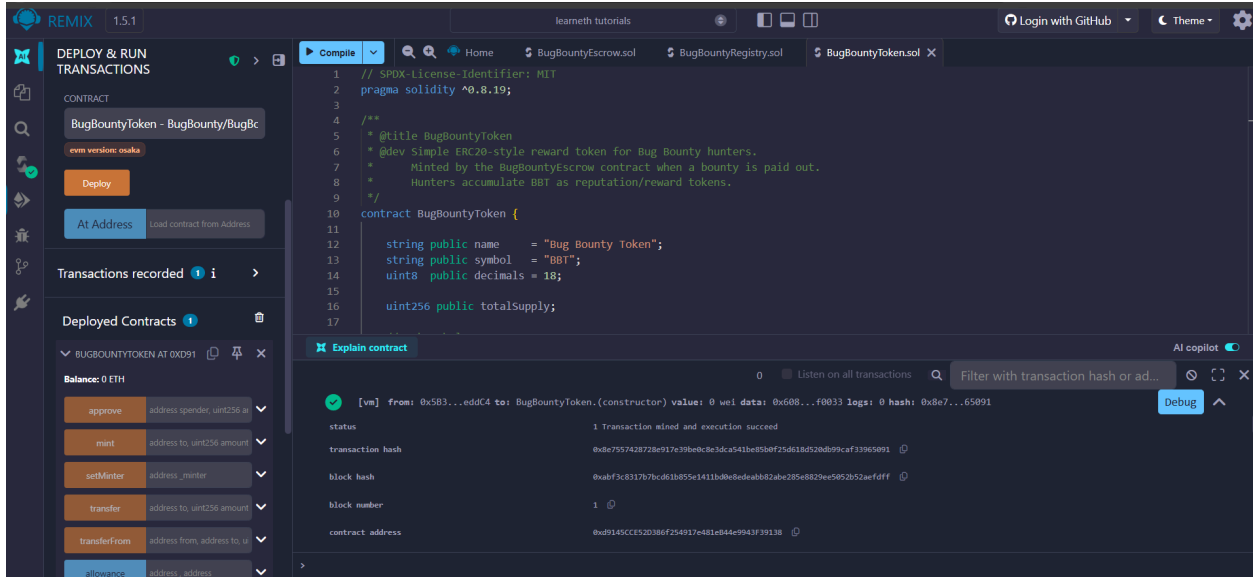
In the **Contract** dropdown, select BugBountyRegistry

Constructor argument:

- `_escrowContract`: paste the BugBountyEscrow address from Step 4

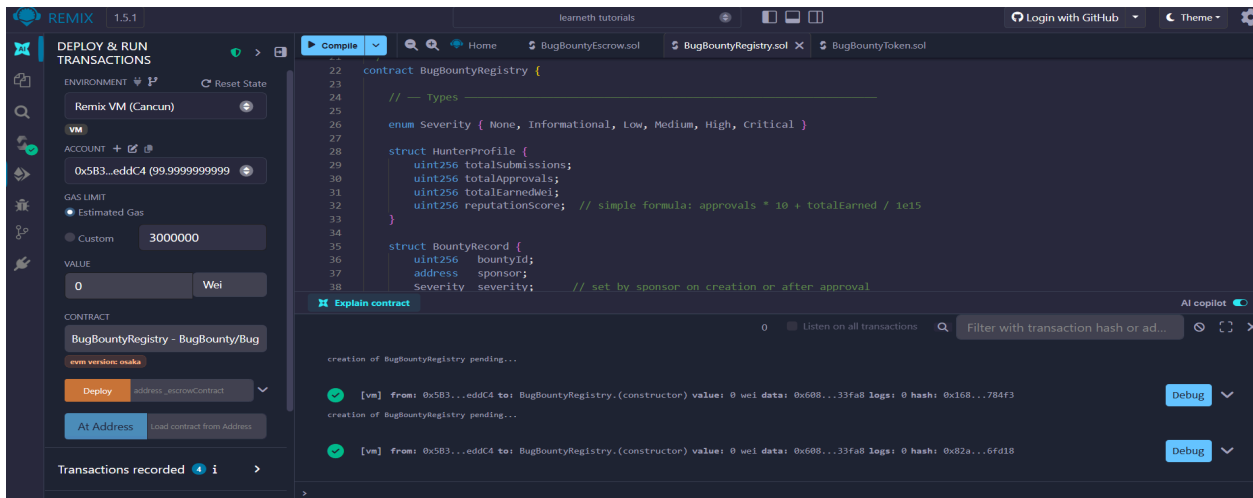
Click **Deploy**

Copy the BugBountyRegistry deployed address



Step 4:- Paste the BugBountyEscrow address to mint BBT tokens then click the transact button

- Expand the deployed BugBountyToken contract in the left panel
- Find the setMinter function
- Paste the BugBountyEscrow address
- Click **transact**



Step 5:- Continuing from the step 4

Fill in that exactly like this

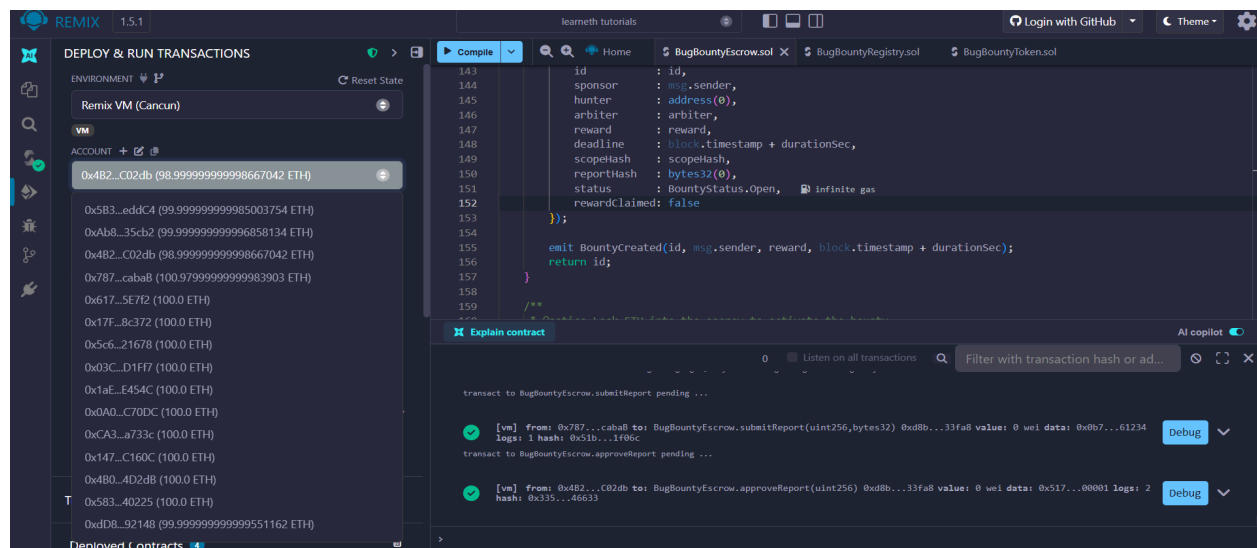
reward → 1000000000000000000

durationSec → 2592000

arbiter → [paste Account 1 address]

scopeHash → 0xabcd....

Step 9:- Register bounty in the registry, then there are 3 people in the accounts part there are 10 fake accounts, then one owner at the top, then there is sponsor, the sponsor will create bounty, sponsor then locks the bounty, hunter submits their bug report and the sponsor approves and then the hunter gets paid, **then the value of that hunter raises**



Conclusion

The Decentralized Bug Bounty Escrow System demonstrates how blockchain technology can solve real trust problems in the cybersecurity industry. By replacing middlemen with smart contracts, it ensures that hunters always get paid fairly and sponsors only release funds when satisfied — all enforced automatically by code, not promises.

With three interconnected contracts handling payments, reputation, and dispute resolution, this system creates a transparent and tamper-proof environment where security researchers can work with confidence. The addition of BBT reward tokens further incentivizes hunters by building their on-chain reputation with every successful submission.

This project proves that decentralized systems are not just a concept — they are practical, deployable tools that can reshape how industries handle trust, payments, and accountability. As Web3 continues to grow, platforms like this represent the future of fair and transparent collaboration between companies and the security community.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab

Experiment 07

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Roll No.	54
Name	Arnav Sawant
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

EXPERIMENT 7

AIM: Implement the embedding wallet (Metamask) and transaction using Solidity

Theory:

Blockchain is a decentralized and distributed digital ledger that securely records transactions across multiple systems. It ensures transparency, security, and immutability, meaning that once data is recorded, it cannot be altered.

Ethereum is a widely used blockchain platform that enables the execution of smart contracts. Smart contracts are self-executing programs written in Solidity that automatically carry out actions when predefined conditions are met, eliminating the need for intermediaries.

MetaMask is a cryptocurrency wallet and browser extension used to interact with blockchain networks. It securely stores user accounts and private keys, and allows users to send, receive, and sign transactions. It also serves as a bridge between Remix IDE and blockchain networks.

Test networks such as Sepolia and RSK Testnet are used for development and testing. They provide free tokens, enabling developers to deploy and test smart contracts without using real cryptocurrency, thereby reducing financial risk.

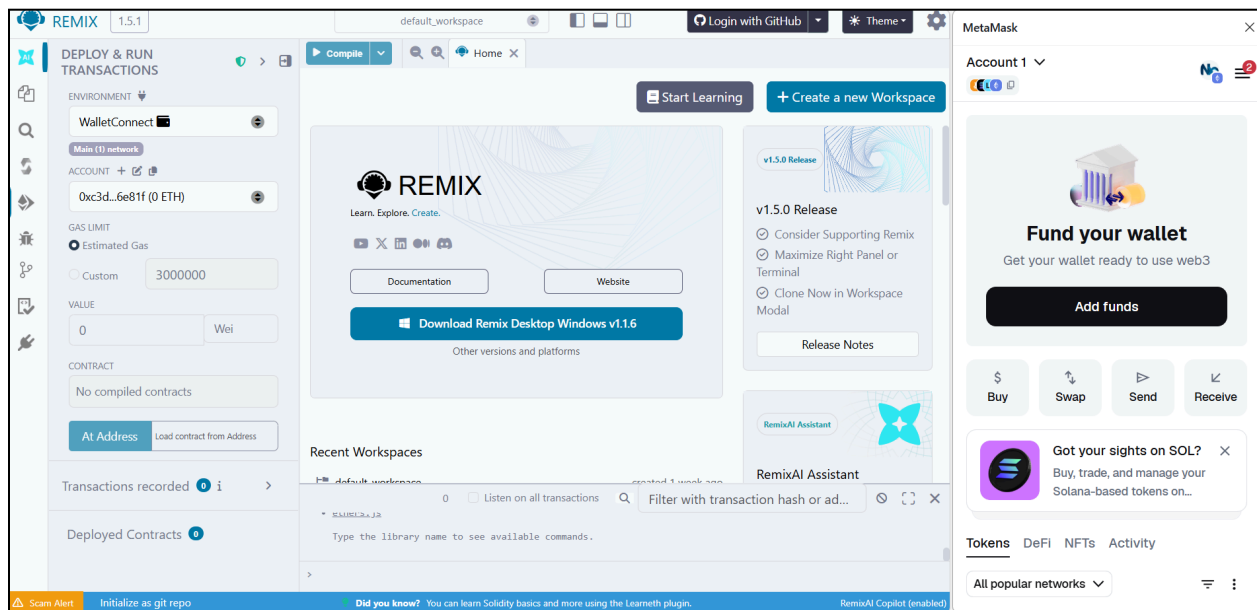
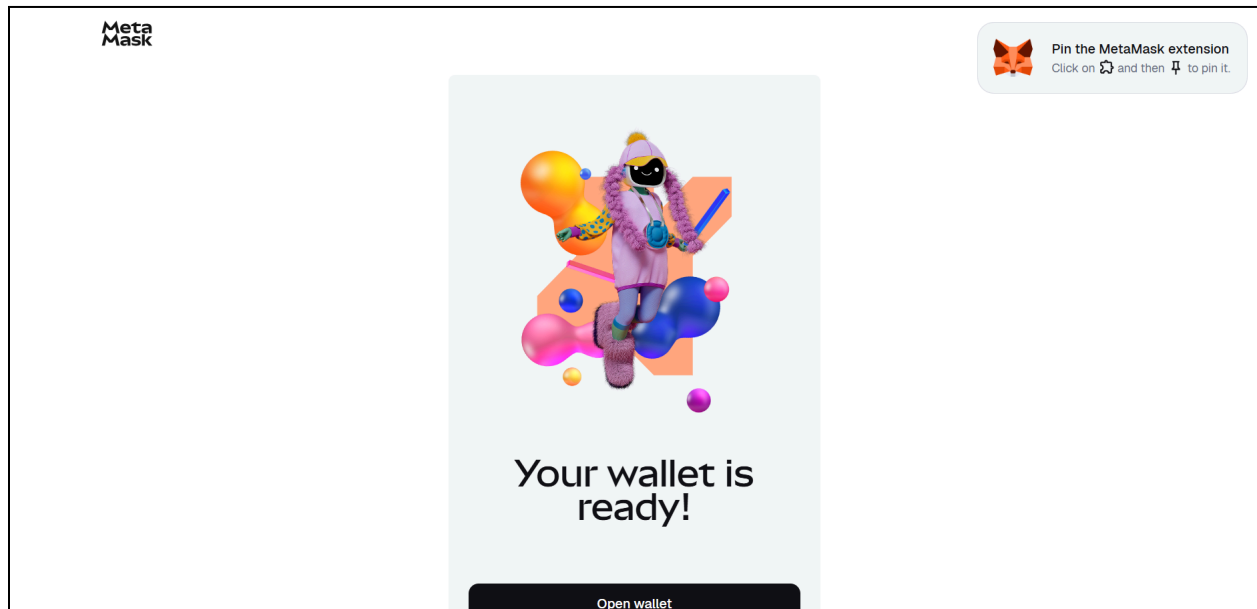
Remix IDE is a web-based development environment used to write, compile, deploy, and test smart contracts. It offers a user-friendly interface and integrates directly with MetaMask for deploying contracts on blockchain networks.

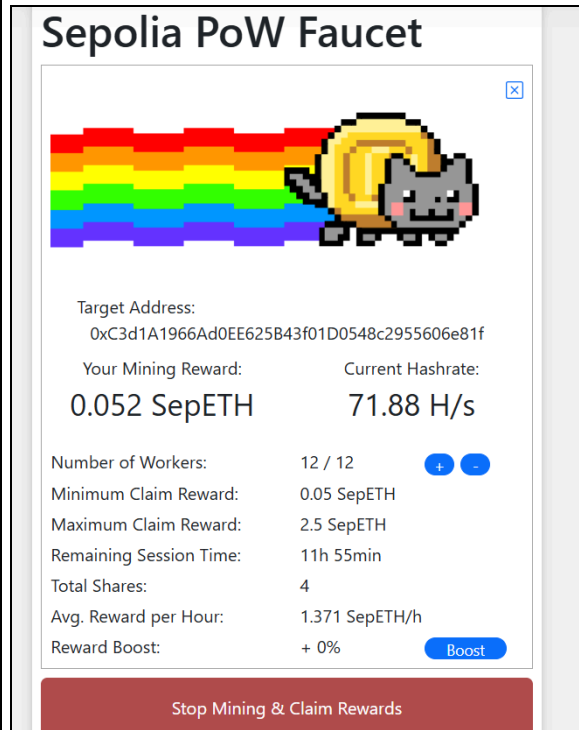
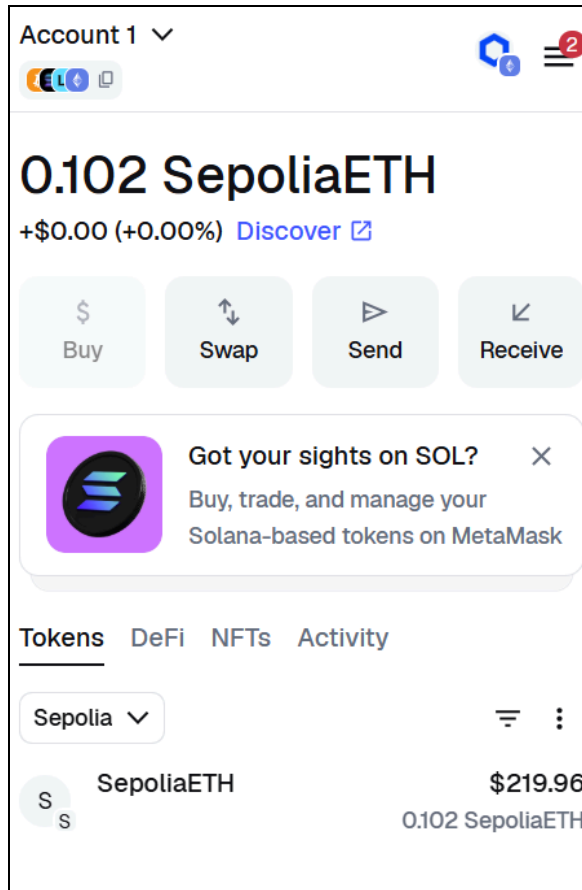
The smart contract deployment process involves writing the contract in Solidity, compiling it to eliminate errors, connecting MetaMask to Remix, and deploying the contract on a testnet by paying gas fees with test tokens.

A blockchain transaction refers to any interaction with the network, such as deploying a contract or executing its functions. Each transaction requires authentication through a digital signature and is permanently recorded on the blockchain. These transactions can be verified using blockchain explorers.

Step 1 —Install MetaMask

Install Metamask and create wallet





Step 2 — Deploy BugBountyToken First

1. In Remix, go to the **Deploy tab** (Ethereum icon on left)
2. Make sure Environment is "**Injected Provider - MetaMask**"
3. In the **Contract dropdown**, select "**BugBountyToken**"
4. Click **Deploy**
5. MetaMask will pop up → click **Confirm**
6. Wait a few seconds → you'll see it appear under "**Deployed Contracts**"
7. **Copy and save its address** (click the copy icon next to it)

REMITX1.5.1

default_workspace

Login with GitHubTheme

DEPLOY & RUN TRANSACTIONS

ACCOUNT0xc3d...6e81f (0 ETH)

GAS LIMIT

Estimated Gas

Custom3000000

VALUE

0Wei

CONTRACT

BugBountyToken - BugBountyToken.s

even version: osaka

Verify Contract on Explorers

Deploy & Verify

At AddressLoad contract from Address

Transactions recorded1

Deployed Contracts0

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

917

918

919

920

921

922

923

924

925

926

927

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

1001

1002

1003

1004

1005

1006

1007

1008

1009

1010

1011

1012

1013

1014

1015

1016

1017

1018

1019

1020

1021

1022

1023

1024

1025

1026

1027

1028

1029

1030

1031

1032

1033

1034

1035

1036

1037

1038

1039

1040

1041

1042

1043

1044

1045

1046

1047

1048

1049

1050

1051

1052

1053

1054

1055

1056

1057

1058

1059

1060

1061

1062

1063

1064

1065

1066

1067

1068

1069

1070

1071

1072

1073

1074

1075

1076

1077

1078

1079

1080

1081

1082

1083

1084

1085

1086

1087

1088

1089

1090

1091

1092

1093

1094

1095

1096

1097

1098

1099

1100

1101

1102

1103

1104

1105

1106

1107

1108

1109

1110

1111

1112

1113

1114

1115

1116

1117

1118

1119

1120

1121

1122

1123

1124

1125

1126

1127

1128

1129

1130

1131

1132

1133

1134

1135

1136

1137

1138

1139

1140

1141

1142

1143

1144

1145

1146

1147

1148

1149

1150

1151

1152

1153

1154

1155

1156

1157

1158

1159

1160

1161

1162

1163

1164

1165

1166

1167

1168

1169

1170

1171

1172

1173

1174

1175

1176

1177

1178

1179

1180

1181

1182

1183

1184

1185

1186

1187

1188

1189

1190

1191

1192

1193

1194

1195

1196

1197

1198

1199

1200

1201

1202

1203

1204

1205

1206

1207

1208

1209

1210

1211

1212

1213

1214

1215

1216

1217

1218

1219

1220

1221

1222

1223

1224

1225

1226

1227

1228

1229

1230

1231

1232

1233

1234

1235

1236

1237

1238

1239

1240

1241

1242

1243

1244

1245

1246

1247

1248

1249

1250

1251

1252

1253

1254

1255

1256

1257

1258

1259

1260

1261

1262

1263

1264

1265

1266

1267

1268

1269

1270

1271

1272

1273

1274

1275

1276

1277

1278

1279

1280

1281

1282

1283

1284

1285

1286

1287

1288

1289

1290

1291

1292

1293

1294

1295

1296

1297

1298

1299

1300

1301

1302

1303

1304

1305

1306

1307

1308

1309

1310

1311

1312

1313

1314

1315

1316

1317

1318

1319

1320

1321

1322

1323

1324

1325

1326

1327

1328

1329

1330

1331

1332

1333

1334

1335

1336

1337

1338

1339

1340

1341

1342

1343

1344

1345

1346

1347

1348

1349

1350

1351

1352

1353

1354

1355

1356

1357

1358

1359

1360

1361

1362

1363

1364

1365

1366

1367

1368

1369

1370

1371

1372

1373

1374

1375

1376

1377

1378

1379

1380

1381

1382

1383

1384

1385

1386

1387

1388

1389

1390

1391

1392

1393

1394

1395

1396

1397

1398

1399

1400

1401

1402

1403

1404

1405

1406

1407

1408

1409

1410

1411

1412

1413

1414

1415

1416

1417

1418

1419

1420

1421

1422

1423

1424

1425

1426

1427

1428

1429

1430

1431

1432

1433

1434

1435

1436

1437

1438

1439

1440

1441

1442

1443

1444

1445

1446

1447

1448

1449

1450

1451

1452

1453

1454

1455

1456

1457

1458

1459

1460

1461

1462

1463

1464

1465

1466

1467

1468

1469

1470

1471

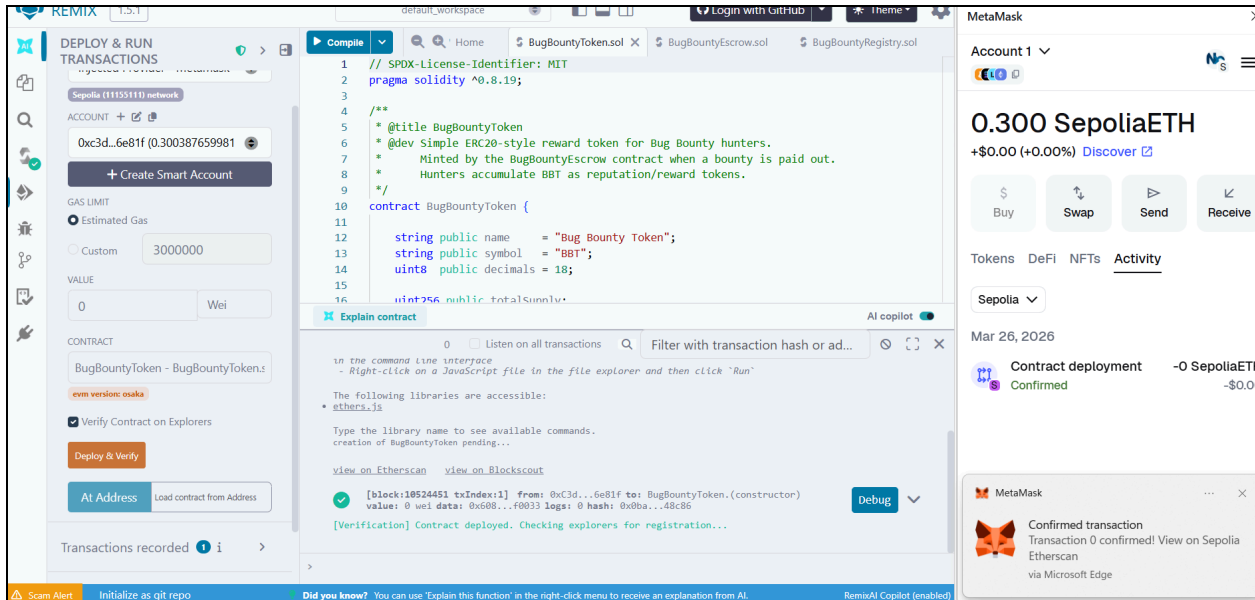
1472

1473

1474

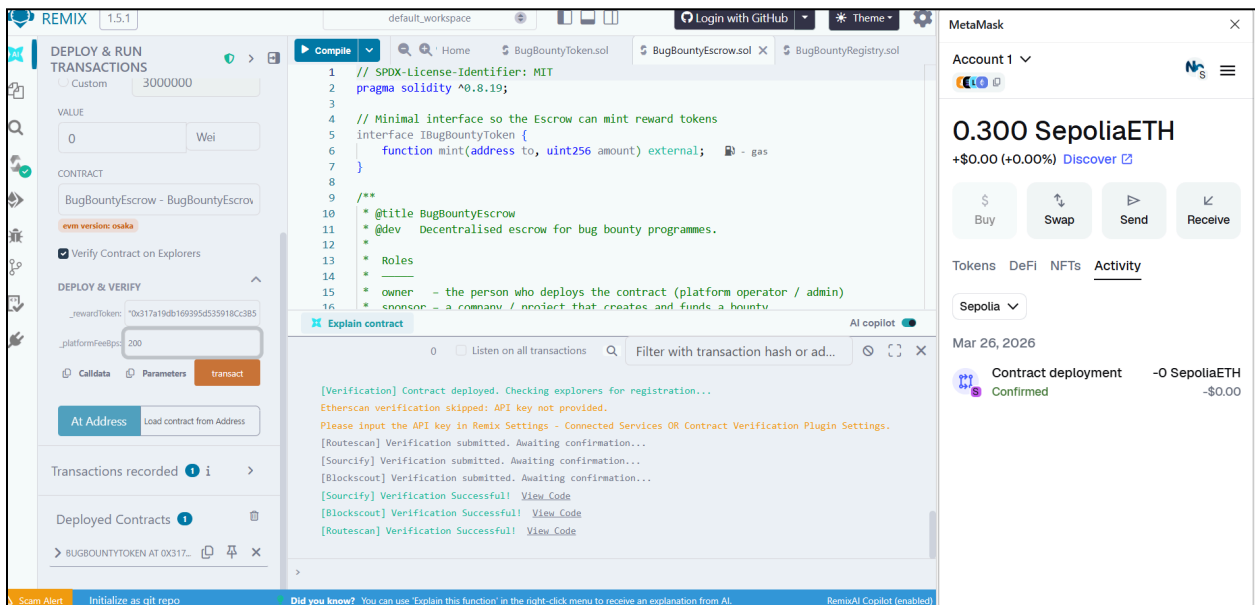
1475

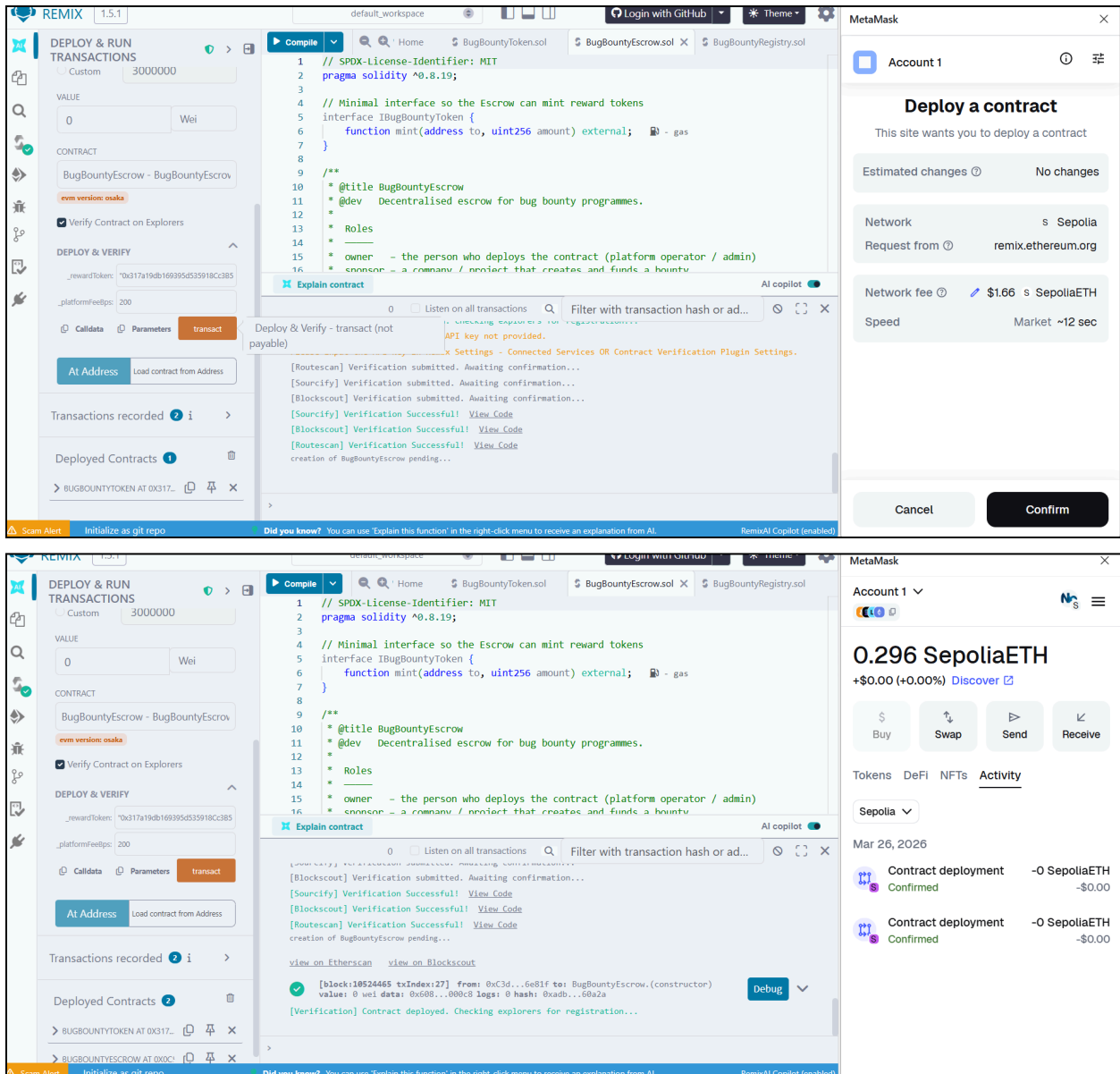
1476</



Step 3 — Deploy BugBountyEscrow

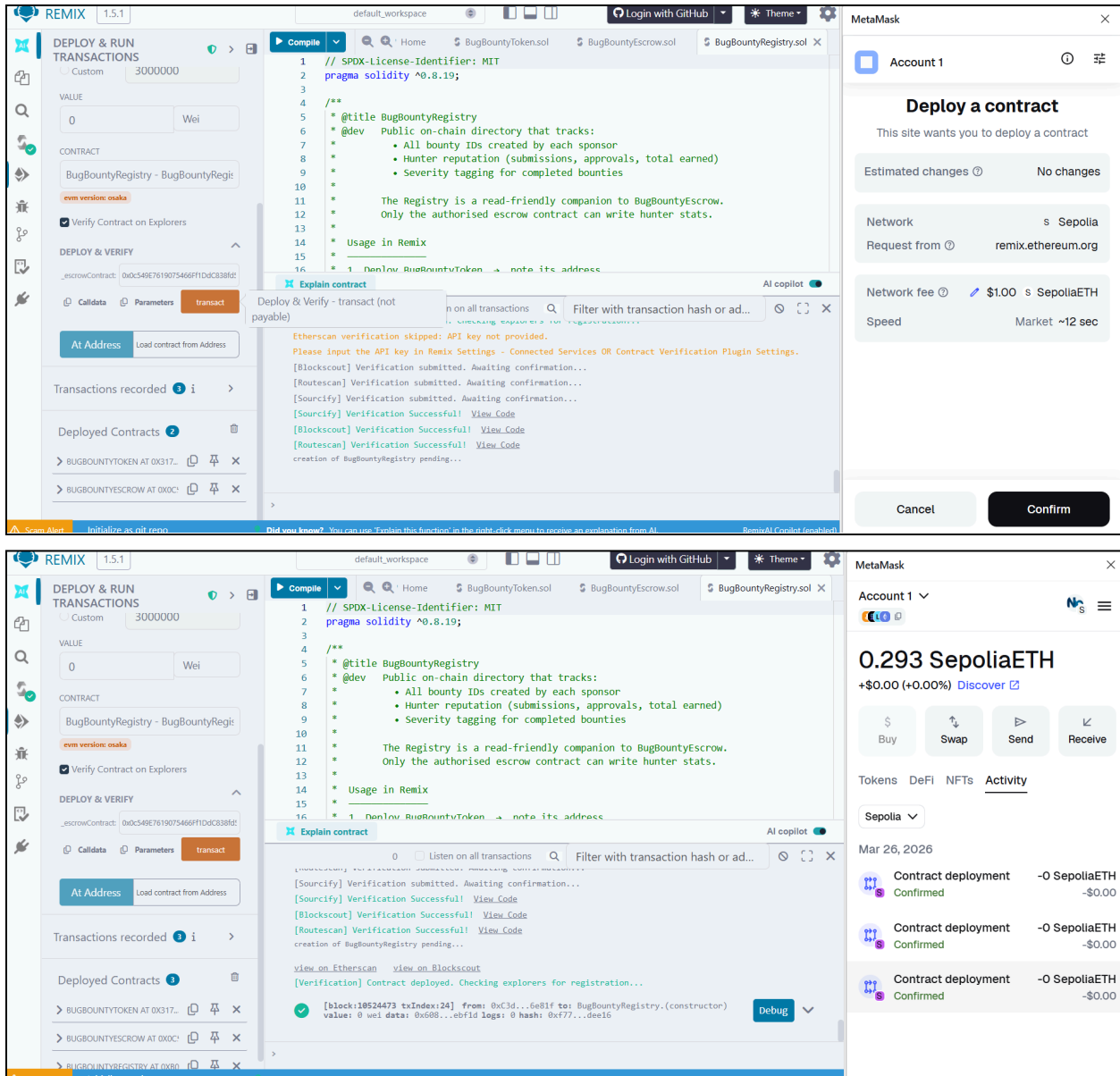
1. In the Contract dropdown, select "**BugBountyEscrow**"
2. You'll see two input fields before deploying:
 - `_rewardToken` → paste the **BugBountyToken** address from Step 1
 - `_platformFeeBps` → type **200** (means 2% fee)
3. Click **Deploy** → Confirm in MetaMask
4. **Copy and save its address**





Step 4 — Deploy BugBountyRegistry

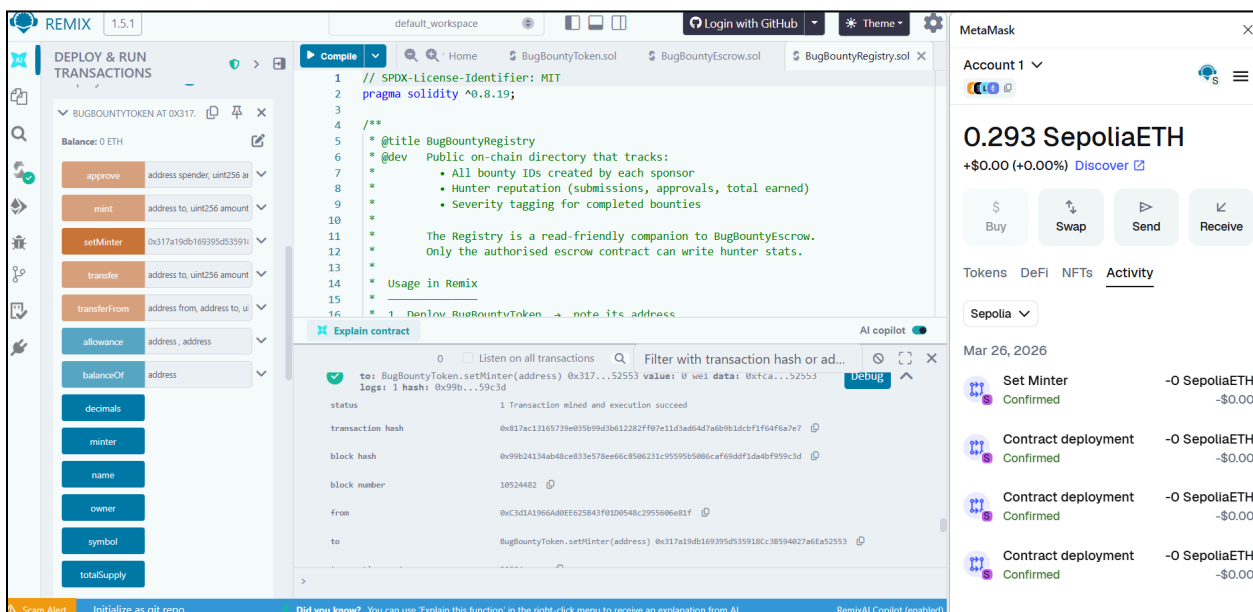
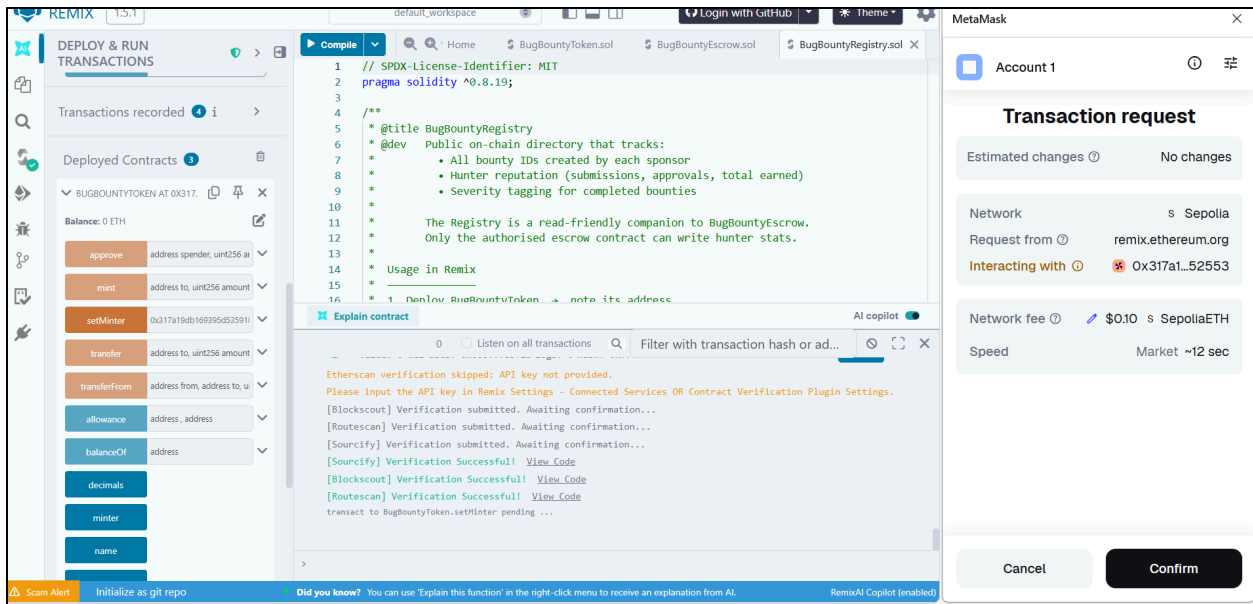
1. Select "**BugBountyRegistry**" in the dropdown
2. Input field:
 - `_escrowContract` → paste the **BugBountyEscrow** address from Step 2
3. Click **Deploy** → Confirm in MetaMask
4. **Copy and save its address**



Step 5 — Link the Contracts Together

On BugBountyToken:

1. Expand it under Deployed Contracts
2. Find **setMinter** function
3. Paste the **BugBountyEscrow** address → click it → Confirm in MetaMask



Conclusion

The experiment successfully demonstrated MetaMask integration with Remix IDE and execution of blockchain transactions. It provided hands-on experience with smart contracts, ensuring secure and transparent interactions on a testnet



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 08

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Roll No.	54
Name	Arnav Sawant
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Blockchain Lab

EXP 8

Aim - To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Theory - Ganache is a personal Ethereum blockchain used for testing and development. It allows developers to deploy contracts, develop decentralized applications (DApps), and run tests in a safe and deterministic environment.

Ganache provides:

- Instant mining of blocks
- Pre-funded test accounts
- Transaction history view
- Graphical interface to monitor blockchain activities

It is part of the Truffle Suite and is commonly used with Remix and Metamask for smart contract deployment.

Steps to Connect Ganache with Metamask and Remix IDE:

1. Install Ganache:
 - Download and install Ganache from the Truffle Suite website.
 - Launch it to start a local blockchain network with test Ether accounts.
2. Connect Ganache Accounts with Metamask:
 - Open Metamask and switch to a Custom RPC Network.
 - Enter the Ganache RPC server URL (e.g., [HTTP://127.0.0.1:7545](http://127.0.0.1:7545)).
 - Import private keys of Ganache accounts into Metamask.
3. Connect Remix IDE with Metamask:
 - Open Remix IDE in the browser.
 - Choose Injected Web3 as the environment in the “Deploy & Run Transactions” tab.
 - Remix will automatically connect to the Metamask account (linked with Ganache).
4. Create a Simple Solidity Smart Contract:
 - Write a Solidity program related to your mini project (e.g., Student Record, Donation Tracker, etc.).

```
pragma solidity ^0.8.0;
contract SimpleStorage {
    uint public data;
    function set(uint x) public {
        data = x;
    }
}
```

5. Compile and Deploy the Contract:
Compile using Remix.
 - Deploy via the Metamask account connected to Ganache.
 - Approve the transaction in Metamask.
6. Check Transaction Details on Ganache:
 - View mined transactions, block details, and balances in the Ganache interface.
7. Interact with the Smart Contract:
 - Use Remix to call contract functions (`set()` or `get()`).
 - Observe the state changes and corresponding transactions on Ganache.

Code -

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Escrow {

    address public buyer;
    address payable public seller;
    address public arbiter;

    bool public buyerApproved;
    bool public sellerApproved;

    constructor(address payable _seller, address _arbiter) payable {
        buyer = msg.sender;
        seller = _seller;
        arbiter = _arbiter;
    }

    function approve() public {
        require(msg.sender == buyer || msg.sender == seller, "Not authorized");

        if (msg.sender == buyer) {
            buyerApproved = true;
        }

        if (msg.sender == seller) {
            sellerApproved = true;
        }

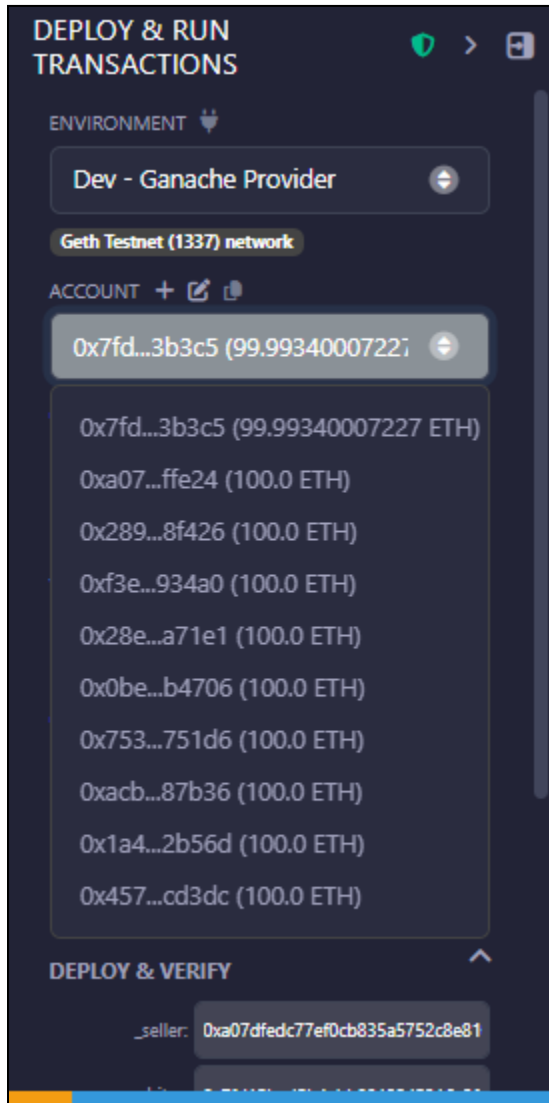
        if (buyerApproved && sellerApproved) {

            uint amount = address(this).balance;

            (bool success, ) = seller.call{value: amount}("");
```

```
        require(success, "Transfer failed");
    }
}
```

Output -



Add wallet

Private key

Imported private keys are backed up to your account and sync automatically when you sign in with the same Google or Apple login.
[Learn more](#) about how imported keys work

Select type

Private key ▾

Enter your private key string here:

.....|

Cancel

Import

ACCOUNT INFORMATION

ACCOUNT ADDRESS

0xA07DFedc77EF0cB835a5752c8e8108E7cA4ffe24

PRIVATE KEY

0xfeb942cbaeb4cc850a33f5c87a3e5af51e76d998538bff2fad3f0c643128ed24

Do not use this private key on a public blockchain; use it for development purposes only!

DONE

ACCOUNTS

BLOCKS

TRANSACTIONS

CONTRACTS

EVENTS

LOGS

SEARCH FOR BLOCK NUMBERS OR TX HASHES

CURRENT BLOCK
2

GAS PRICE
20000000000

GAS LIMIT
6721975

HARDFORK
MERGE

NETWORK ID
5777

RPC SERVER
HTTP://127.0.0.1:7545

MINING STATUS
AUTOMINING

WORKSPACE
QUICKSTART

SAVE

SWITCH

BLOCK
2

MINED ON
2026-03-17 11:46:30

GAS USED
3000000

1 TRANSACTION

BLOCK
1

MINED ON
2026-03-17 11:46:19

GAS USED
3000000

1 TRANSACTION

BLOCK
0

MINED ON
2026-03-17 11:36:01

GAS USED
0

NO TRANSACTIONS

CURRENT BLOCK
2

GAS PRICE
20000000000

GAS LIMIT
6721975

HARDFORK
MERGE

NETWORK ID
5777

RPC SERVER
HTTP://127.0.0.1:7545

MINING STATUS
AUTOMINING

WORKSPACE
QUICKSTART

SAVE

SWITCH

TX HASH

0xb85506ee1d0385b8a59d33a0e97cbdc501667e7d45ef7ac1f5beec6aa9a79243

CONTRACT CREATION

FROM ADDRESS

0x7fd15bAd5b4AbB0912345316A3fcD453B2a3b3C5

CREATED CONTRACT ADDRESS

0x7b10fEE11188F75915ce12A07120d8442A7638af

GAS USED

3000000

VALUE

0

TX HASH

0xa4f74fed3fc1e12bcc5777e0337466b545001a6ecac86fdea5c9e04f4f7aa085

CONTRACT CREATION

FROM ADDRESS

0x7fd15bAd5b4AbB0912345316A3fcD453B2a3b3C5

CREATED CONTRACT ADDRESS

0x018b4fb3336095E9cdf9988B0d8f8C7D22e24ECC

GAS USED

3000000

VALUE

0

Conclusion - Solidity smart contract were deployed on remix IDE and transaction were executed successfully on ganache blockchain with the help of metamask wallets.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 09

Aim: To implement a Private Ethereum Blockchain using Geth.

Roll No.	54
Name	Arnav Sawant
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Blockchain Lab

EXP 9

Aim - To implement a Private Ethereum Blockchain using Geth.

Theory -

Ethereum is an open-source, decentralized blockchain platform that enables developers to build and deploy **smart contracts** and **decentralized applications (DApps)**. Unlike public blockchains such as the Ethereum mainnet, a **Private Ethereum Blockchain** is restricted to specific participants, making it ideal for **development, testing, and enterprise applications** where confidentiality, control, and performance are essential.

Geth (Go-Ethereum) is one of the most widely used Ethereum clients, written in the Go programming language. It allows users to run an Ethereum node, mine Ether, and interact with the blockchain through **JSON-RPC, command-line, or JavaScript consoles**.

Setting up a **Private Ethereum Network** involves creating a customized blockchain environment isolated from the public Ethereum network. It enables developers to test smart contracts, consensus mechanisms, and network behavior without incurring gas costs or depending on public validators.

Steps Involved:

1. **Choosing a Network ID:**
2. Each private blockchain must have a unique network ID to differentiate it from public or other private networks.
3. **Choosing a Consensus Algorithm:**
4. Ethereum supports different consensus mechanisms such as **Proof of Work (PoW)** and **Proof of Authority (PoA)**. In private networks, **Clique (PoA)** is often used for faster block confirmation.
5. **Creating a Genesis Block:**
6. The genesis block is the first block of the blockchain, defining network parameters such as difficulty, gas limit, initial account balances, and consensus settings.
7. **Initializing the Geth Database:**
8. The `geth init` command initializes the blockchain using the `genesis.json` file, preparing the data directory for node operation.
9. **Setting up Networking:**
10. Nodes communicate using **enodes** (Ethereum Node URLs) that contain IP addresses and public keys for peer discovery and connection.
11. **Running the Member Nodes:**

12. Each participant runs a Geth node that connects to the private network and participates in block validation and transaction processing.
13. **Running a Signer (in Clique):**
14. In the **Clique consensus**, designated **signers** validate and sign blocks. These accounts are pre-configured in the genesis file to ensure trusted block production.

Outputs -

1. First install all the necessary dependencies like curl, git, build-essential.

```
sudo apt install wget curl git build-essential -y
```

```
deilsy@deilsy:~$ sudo apt install wget curl git build-essential -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
wget is already the newest version (1.21.4-1ubuntu4.1).
wget set to manually installed.
git is already the newest version (1:2.43.0-1ubuntu7.3).
build-essential is already the newest version (12.10ubuntu1).
The following additional packages will be installed:
  libcurl3t64-gnutls libcurl4t64
The following packages will be upgraded:
  curl libcurl3t64-gnutls libcurl4t64
3 upgraded, 0 newly installed, 0 to remove and 57 not upgraded.
Need to get 902 kB of archives.
After this operation, 0 B of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 curl amd64 8
.5.0-2ubuntu10.8 [227 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libcurl4t64
amd64 8.5.0-2ubuntu10.8 [342 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libcurl3t64-
gnutls amd64 8.5.0-2ubuntu10.8 [334 kB]
Fetched 902 kB in 1s (650 kB/s)
(Reading database ... 46694 files and directories currently installed.)
Preparing to unpack .../curl_8.5.0-2ubuntu10.8_amd64.deb ...
Unpacking curl (8.5.0-2ubuntu10.8) over (8.5.0-2ubuntu10.6) ...
Preparing to unpack .../libcurl4t64_8.5.0-2ubuntu10.8_amd64.deb ...
Unpacking libcurl4t64:amd64 (8.5.0-2ubuntu10.8) over (8.5.0-2ubuntu10.6) ...
Preparing to unpack .../libcurl3t64-gnutls_8.5.0-2ubuntu10.8_amd64.deb ...
Unpacking libcurl3t64-gnutls:amd64 (8.5.0-2ubuntu10.8) over (8.5.0-2ubuntu10
.6) ...
Setting up libcurl4t64:amd64 (8.5.0-2ubuntu10.8) ...
Setting up libcurl3t64-gnutls:amd64 (8.5.0-2ubuntu10.8) ...
Setting up curl (8.5.0-2ubuntu10.8) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
```

2. Install geth using wget

```
cd /usr/local/bin && sudo wget
```

```
https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.12.0-e5
01b3b0.tar.gz
```

```

deilsy@deilsy:~$ cd /usr/local/bin && sudo wget https://gethstore.blob.core.w
indows.net/builds/geth-linux-amd64-1.12.0-e501b3b0.tar.gz
--2026-03-26 03:54:03-- https://gethstore.blob.core.windows.net/builds/geth
-linux-amd64-1.12.0-e501b3b0.tar.gz
Resolving gethstore.blob.core.windows.net (gethstore.blob.core.windows.net).
.. 20.60.40.164
Connecting to gethstore.blob.core.windows.net (gethstore.blob.core.windows.n
et)|20.60.40.164|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 26856444 (26M) [application/octet-stream]
Saving to: 'geth-linux-amd64-1.12.0-e501b3b0.tar.gz'

geth-linux-amd64-1 100%[=====>] 25.61M 734KB/s in 27s

2026-03-26 03:54:32 (968 KB/s) - 'geth-linux-amd64-1.12.0-e501b3b0.tar.gz' s
aved [26856444/26856444]

deilsy@deilsy:/usr/local/bin$ ls
geth-linux-amd64-1.12.0-e501b3b0.tar.gz
deilsy@deilsy:/usr/local/bin$ █

```

3. Extract the tar file and move the geth into /usr/local/bin/geth path & check the geth version

```

sudo tar -xvf geth-linux-amd64-1.12.0-e501b3b0.tar.gz
sudo mv geth-linux-amd64-1.12.0-e501b3b0/geth /usr/local/bin/geth
geth version

```

```

deilsy@deilsy:/usr/local/bin$ sudo mv geth-linux-amd64-1.12.0-e501b3b0/geth /
usr/local/bin/geth
deilsy@deilsy:/usr/local/bin$ geth version
Geth
Version: 1.12.0-stable
Git Commit: e501b3b05db8e169f67dc78b7b59bc352b3c638d
Git Commit Date: 20230525
Architecture: amd64
Go Version: go1.20.3
Operating System: linux
GOPATH=
GOROOT=
deilsy@deilsy:/usr/local/bin$ █

```

4. Create a folder named private_ethereum_setup and create another 2 directories naming node1 and node2 individually inside private_ethereum_setup directory.

```

mkdir private_ethereum_setup
cd private_ethereum_setup && mkdir node1 node2

```

```

deilsy@deilsy:~$ ls private_ethereum_setup/
node1  node2
deilsy@deilsy:~$ █

```

5. Check the files

6. Create account for node1

```
geth --datadir "node1" account new
```

```
deilsy@deilsy:~/private_ethereum_setup$ geth --datadir "node1" account new
INFO [03-26|04:01:54.815] Maximum peer count          ETH=50 LE
S=0 total=50
INFO [03-26|04:01:54.816] Smartcard socket not found, disabling err="stat
/run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not f
orget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x563A5a670260417B163AD1696fCBfA6849906d06
Path of the secret key file: node1/keystore/UTC--2026-03-26T04-02-01.7653580
44Z--563a5a670260417b163ad1696fcbfa6849906d06

- You can share your public address with anyone. Others need it to interact
with you.
- You must NEVER share the secret key with anyone! The key controls access t
o your funds!
- You must BACKUP your key file! Without the key, it's impossible to access
account funds!
- You must REMEMBER your password! Without the password, it's impossible to
decrypt the key!

deilsy@deilsy:~/private_ethereum_setup$ █
```

7. Do the same for node2

```
geth --datadir "node2" account new
```

```
deilsy@deilsy:~/private_ethereum_setup$ geth --datadir "node2" account new
INFO [03-26|04:02:43.813] Maximum peer count          ETH=50 LE
S=0 total=50
INFO [03-26|04:02:43.813] Smartcard socket not found, disabling err="stat
/run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not f
orget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x6d04eA66725A40F5669E2d6f4f77eb4Ecf756a3f
Path of the secret key file: node2/keystore/UTC--2026-03-26T04-02-49.2597870
89Z--6d04ea66725a40f5669e2d6f4f77eb4ecf756a3f

- You can share your public address with anyone. Others need it to interact
with you.
- You must NEVER share the secret key with anyone! The key controls access t
o your funds!
- You must BACKUP your key file! Without the key, it's impossible to access
account funds!
- You must REMEMBER your password! Without the password, it's impossible to
decrypt the key!
```

8. Create a file called genesis.json in the same directory and copy paste these contents

```
{
```

```

"config": {
  "chainId": 12345,
  "homesteadBlock": 0,
  "eip150Block": 0,
  "eip155Block": 0,
  "eip158Block": 0,
  "byzantiumBlock": 0,
  "constantinopleBlock": 0,
  "petersburgBlock": 0,
  "istanbulBlock": 0,
  "muirGlacierBlock": 0,
  "berlinBlock": 0,
  "londonBlock": 0,
  "arrowGlacierBlock": 0,
  "grayGlacierBlock": 0,
  "clique": {
    "period": 5,
    "epoch": 30000
  }
},
"difficulty": "1",
"gasLimit": "8000000000",
"extradata":
"0x0000000000000000000000000000000000000000000000000000000000000000
0000000098608ADf9c785d54f40cDcf6700E990771b1922600000000000000
0000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000
0",
  "alloc": {
    "0x98608ADf9c785d54f40cDcf6700E990771b19226": {
"balance": "1000000000000000000000000" },
    "0x7B25e791D24A3F5c453A9E5468cF6cEa2243092C": {
"balance": "1200000000000000000000000000" }
  }
}

```

Make changes in the alloc part replace the public keys of the account that you have generated in earlier steps.

```

{
  "config": {
    "chainId": 12345,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,
    "constantinopleBlock": 0,
    "petersburgBlock": 0,
    "istanbulBlock": 0,
    "muirGlacierBlock": 0,
    "berlinBlock": 0,
    "londonBlock": 0,
    "arrowGlacierBlock": 0,
    "grayGlacierBlock": 0,
    "clique": {
      "period": 5,
      "epoch": 30000
    }
  },
  "difficulty": "1",
  "gasLimit": "8000000000",
  "extradata": "0x0000000000000000000000000000000000000000000000000000000000000000>",
  "alloc": {
    "0x563A5a670260417B163AD1696fCBfA6849906d06": { "balance": "100000000000>" },
    "0x6d04eA66725A40F5669E2d6f4f77eb4Ecf756a3f": { "balance": "120000000000>" }
  }
}

```

9. Initializing geth for node1 using the genesis.json file

geth init --datadir node1 genesis.json

```

deilsy@deilsy:~/private_ethereum_setup$ geth init --datadir node1 genesis.js
n
INFO [03-26|04:06:53.763] Maximum peer count                      ETH=50 LE
S=0 total=50
INFO [03-26|04:06:53.764] Smartcard socket not found, disabling   err="stat
/run/pcscd/pcscd.comm: no such file or directory"
INFO [03-26|04:06:53.767] Set global gas cap                      cap=50,00
0,000
INFO [03-26|04:06:53.768] Initializing the KZG library            backend=g
okzg
INFO [03-26|04:06:53.810] Defaulting to pebble as the backing database
INFO [03-26|04:06:53.810] Allocated cache and file handles       database=
/home/deilsy/private_ethereum_setup/node1/geth/chaindata cache=16.00MiB hand
les=16
INFO [03-26|04:06:53.957] Opened ancient database                 database=
/home/deilsy/private_ethereum_setup/node1/geth/chaindata/ancient/chain reado
nly=false
INFO [03-26|04:06:53.957] Writing custom genesis block
INFO [03-26|04:06:53.958] Persisted trie from memory database    nodes=3 s
ize=413.00B time="100.853µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 li
vesize=0.00B
INFO [03-26|04:06:53.976] Successfully wrote genesis state        database=
chaindata hash=732ee4..dabc1f
INFO [03-26|04:06:53.976] Defaulting to pebble as the backing database
INFO [03-26|04:06:53.976] Allocated cache and file handles       database=
/home/deilsy/private_ethereum_setup/node1/geth/lightchaindata cache=16.00MiB
handles=16
INFO [03-26|04:06:54.074] Opened ancient database                 database=
/home/deilsy/private_ethereum_setup/node1/geth/lightchaindata/ancient/chain
readonly=false
INFO [03-26|04:06:54.074] Writing custom genesis block
INFO [03-26|04:06:54.074] Persisted trie from memory database    nodes=3 s
ize=413.00B time="18.276µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 li
vesize=0.00B
INFO [03-26|04:06:54.086] Successfully wrote genesis state        database=
lightchaindata hash=732ee4..dabc1f
deilsy@deilsy:~/private_ethereum_setup$ █

```

10. Do the same for node2

```
geth init --datadir node2 genesis.json
```

```

deilsy@deilsy:~/private_ethereum_setup$ geth init --datadir node2 genesis.js
n
INFO [03-26|04:07:16.531] Maximum peer count                      ETH=50 LE
S=0 total=50
INFO [03-26|04:07:16.531] Smartcard socket not found, disabling  err="stat
/run/pcscd/pcscd.comm: no such file or directory"
INFO [03-26|04:07:16.534] Set global gas cap                      cap=50,00
0,000
INFO [03-26|04:07:16.534] Initializing the KZG library            backend=g
okzg
INFO [03-26|04:07:16.580] Defaulting to pebble as the backing database
INFO [03-26|04:07:16.580] Allocated cache and file handles       database=
/home/deilsy/private_ethereum_setup/node2/geth/chaindata cache=16.00MiB hand
les=16
INFO [03-26|04:07:16.713] Opened ancient database                 database=
/home/deilsy/private_ethereum_setup/node2/geth/chaindata/ancient/chain reado
nly=false
INFO [03-26|04:07:16.713] Writing custom genesis block
INFO [03-26|04:07:16.713] Persisted trie from memory database     nodes=3 s
ize=413.00B time="23.604µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 liv
esize=0.00B
INFO [03-26|04:07:16.724] Successfully wrote genesis state        database=
chaindata hash=732ee4..dabc1f
INFO [03-26|04:07:16.724] Defaulting to pebble as the backing database
INFO [03-26|04:07:16.724] Allocated cache and file handles       database=
/home/deilsy/private_ethereum_setup/node2/geth/lightchaindata cache=16.00MiB
handles=16
INFO [03-26|04:07:16.799] Opened ancient database                 database=
/home/deilsy/private_ethereum_setup/node2/geth/lightchaindata/ancient/chain
readonly=false
INFO [03-26|04:07:16.800] Writing custom genesis block
INFO [03-26|04:07:16.800] Persisted trie from memory database     nodes=3 s
ize=413.00B time="29.651µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 liv
esize=0.00B
INFO [03-26|04:07:16.814] Successfully wrote genesis state        database=
lightchaindata hash=732ee4..dabc1f

```

11. Now we will install bootnode for configuring both the nodes.

```

sudo add-apt-repository -y ppa:ethereum/ethereum
sudo apt-get update
sudo apt-get install bootnode

```

12. Create a key for the bootnode and save it to boot.key in the folder

```
bootnode -genkey boot.key
```

```

deilsy@deilsy:~/private_ethereum_setup$ bootnode -genkey boot.key
deilsy@deilsy:~/private_ethereum_setup$ ls
boot.key genesis.json node1 node2
deilsy@deilsy:~/private_ethereum_setup$ █

```

13. Copy this command and copy the enode://.... This will be required later

```
bootnode -nodekey boot.key -verbosity 9 -addr :30305
```

```
deilsy@delsy:~/private_ethereum_setup$ bootnode -nodekey boot.key -verbosity
9 -addr :30305
enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917f51333118e81f54e7
5919746a331d55af7f6edba870a5ef5ed12b60af554ccb922bc77ab41f27@127.0.0.1:0?dis
cport=30305
Note: you're using cmd/bootnode, a developer tool.
We recommend using a regular node as bootstrap node for production deploymen
ts.
INFO [03-26|04:20:52.969] New local node record                      seq=1,774
,498,852,968 id=7212962da899bfc0 ip="invalid IP" udp=0 tcp=0
```

14. Now on another terminal we will start mining node1

```
geth --datadir node1 --port 30306 --bootnodes
enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917f5133
3118e81f54e75919746a331d55af7f6edba870a5ef5ed12b60af554ccb922bc7
7ab41f27@127.0.0.1:0?discport=30305 --networkid 123454321
--unlock 0x563A5a670260417B163AD1696fCBfA6849906d06 --password
node1/password.txt --authrpc.port 8551 --miner.etherbase
0x563A5a670260417B163AD1696fCBfA6849906d06 --mine
```

Replace the address with your own created

```

deilsy@deilsy:~/private_ethereum_setup$ geth --datadir node1 --port 30306 --b
ootnodes enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917f5133311
8e81f54e75919746a331d55af7f6edba870a5ef5ed12b60af554ccb922bc77ab41f27@127.0.
0.1:0?discport=30305 --networkid 123454321 --unlock 0x563A5a670260417B163AD1
696fCBfA6849906d06 --password node1/password.txt --authrpc.port 8551 --miner
.etherbase 0x563A5a670260417B163AD1696fCBfA6849906d06 --mine
INFO [03-26|04:28:57.471] Maximum peer count                      ETH=50 LE
S=0 total=50
INFO [03-26|04:28:57.472] Smartcard socket not found, disabling      err="stat
/run/pcscd/pcscd.comm: no such file or directory"
INFO [03-26|04:28:57.474] Set global gas cap                      cap=50,00
0,000
INFO [03-26|04:28:57.474] Initializing the KZG library                backend=g
okzg
INFO [03-26|04:28:57.529] Allocated trie memory caches              clean=154
.00MiB dirty=256.00MiB
INFO [03-26|04:28:57.529] Using pebble as the backing database
INFO [03-26|04:28:57.530] Allocated cache and file handles          database=
/home/deilsy/private_ethereum_setup/node1/geth/chaindata cache=512.00MiB han
dles=524,288
INFO [03-26|04:28:57.629] Opened ancient database                  database=
/home/deilsy/private_ethereum_setup/node1/geth/chaindata/ancient/chain reado
nly=false
INFO [03-26|04:28:57.641] Initialising Ethereum protocol          network=1
23,454,321 dbversion=8
INFO [03-26|04:28:57.641]
INFO [03-26|04:28:57.641] -----
-----
INFO [03-26|04:28:57.641] Chain ID: 12345 (unknown)
INFO [03-26|04:28:57.641] Consensus: Clique (proof-of-authority)

```

15. Now do same for node2

```

geth --datadir node2 --port 30307 --bootnodes
enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917
f51333118e81f54e75919746a331d55af7f6edba870a5ef5ed12b60af55
4ccb922bc77ab41f27@127.0.0.1:0?discport=30305 --networkid
123454321 --unlock
0x6d04eA66725A40F5669E2d6f4f77eb4Ecf756a3f --password
node2/password.txt --authrpc.port 8552

```

```

deilsy@deilsy:~/private_ethereum_setup$ geth --datadir node2 --port 30307 --b
ootnodes enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917f5133311
8e81f54e75919746a331d55af7f6edba870a5ef5ed12b60af554ccb922bc77ab41f27@127.0.
0.1:0?discport=30305 --networkid 123454321 --unlock 0x6d04eA66725A40F5669E2d
6f4f77eb4Ecf756a3f --password node2/password.txt --authrpc.port 8552
INFO [03-26|04:35:17.744] Maximum peer count                      ETH=50 LE
S=0 total=50
INFO [03-26|04:35:17.745] Smartcard socket not found, disabling    err="stat
/run/pcscd/pcscd.comm: no such file or directory"
INFO [03-26|04:35:17.748] Set global gas cap                      cap=50,00
0,000
INFO [03-26|04:35:17.748] Initializing the KZG library            backend=g
okzg
INFO [03-26|04:35:17.795] Allocated trie memory caches          clean=154
.00MiB dirty=256.00MiB
INFO [03-26|04:35:17.795] Using pebble as the backing database
INFO [03-26|04:35:17.795] Allocated cache and file handles      database=
/home/deilsy/private_ethereum_setup/node2/geth/chaindata cache=512.00MiB han
dles=524,288
INFO [03-26|04:35:17.928] Opened ancient database              database=
/home/deilsy/private_ethereum_setup/node2/geth/chaindata/ancient/chain reado
nly=false
INFO [03-26|04:35:17.929] Initialising Ethereum protocol        network=1
23,454,321 dbversion=<nil>
INFO [03-26|04:35:17.930]
INFO [03-26|04:35:17.930] -----
-----
INFO [03-26|04:35:17.930] Chain ID: 12345 (unknown)
INFO [03-26|04:35:17.930] Consensus: Clique (proof-of-authority)
INFO [03-26|04:35:17.930]
INFO [03-26|04:35:17.930] Pre-Merge hard forks (block based):
INFO [03-26|04:35:17.930]   - Homestead: #0 (https:/
/github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-up
grades/homestead.md)

```

16. Now we will interact with the blockchain using javascript console

```
geth attach node1/geth.ipc
```

```

deilsy@deilsy:~/private_ethereum_setup$ geth attach node1/geth.ipc
Welcome to the Geth JavaScript console!

instance: Geth/v1.12.0-stable-e501b3b0/linux-amd64/go1.20.3
coinbase: 0x563a5a670260417b163ad1696fcbfa6849906d06
at block: 105 (Thu Mar 26 2026 04:37:38 GMT+0000 (UTC))
  datadir: /home/deilsy/private_ethereum_setup/node1
  modules: admin:1.0 clique:1.0 debug:1.0 engine:1.0 eth:1.0 miner:1.0 net:1.
0 rpc:1.0 txpool:1.0 web3:1.0

To exit, press ctrl-d or type exit
> █

```

17. Interacting with the blockchain

```
> net.peerCount
1
> eth.accounts
["0x563a5a670260417b163ad1696fcbfa6849906d06"]
> eth.accounts[0]
"0x563a5a670260417b163ad1696fcbfa6849906d06"
> eth.blockNumber
302
> eth.getBalance(eth.accounts[0])
1e+23
> web3.fromWei(eth.getBalance("0x563A5a670260417B163AD1696fCBfA6849906d06"),
"ether")
100000
```

```

> eth.getBlock(eth.blockNumber)
{
  baseFeePerGas: 7,
  difficulty: 2,
  extraData: "0xd883010c00846765746888676f312e32302e33856c696e75780000000000
000040f57c8f0c1bbe3fa04b217bfdc354aa1e411ae790f2aa93ecf3b9f49ccbfd224b4c30f2
d379e2f80d9626b9aa24660a7e3f8e6276b34aba4825c2956e45a62200",
  gasLimit: 586348211,
  gasUsed: 0,
  hash: "0x5cd8ce4ebaace9c9674c46f1346b87b916bf6e15113dd3270634ac6b5cb0e96c"
,
  logsBloom: "0x0000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000",
  miner: "0x0000000000000000000000000000000000000000000000000000000000",
  mixHash: "0x0000000000000000000000000000000000000000000000000000000000
00",
  nonce: "0x0000000000000000",
  number: 318,
  parentHash: "0x6a5fac551749b1a9ee9a3e7c8b0db9c6b3f4ffd7175e8df285c1bb74b70
d64cd",
  receiptsRoot: "0x56e81f171bcc55a6ff8345e692c0f86e5b48e01b996cad001622fb5e
363b421",
  sha3Uncles: "0x1dcc4de8dec75d7aab85b567b6ccd41ad312451b948a7413f0a142fd40d
49347",
  size: 610,
  stateRoot: "0x121c3db0cbfb3018fc585199bda672875d3caa3a3e83f480387ce86ac9d2
7e34",
  timestamp: 1774500923,
  totalDifficulty: 637,
  transactions: [],
  transactionsRoot: "0x56e81f171bcc55a6ff8345e692c0f86e5b48e01b996cad001622
fb5e363b421",
  uncles: []
}

```

18. Sending transactions between accounts

```

> eth.sendTransaction({from:"0x563A5a670260417B163AD1696fCBfA6849906d06", to
:"0x6d04eA66725A40F5669E2d6f4f77eb4Ec756a3f", value: web3.toWei(10, "ether"
), gas:30000})
"0x3cb6cc9cd65c927b0a0af8351e757ef8fdf76ea00d66b8863b3c9fd7c3657508"
> web3.fromWei(eth.getBalance("0x563A5a670260417B163AD1696fCBfA6849906d06"),
"ether")
99989.99999999999853
> web3.fromWei(eth.getBalance("0x6d04eA66725A40F5669E2d6f4f77eb4Ec756a3f"),
"ether")
120010

```

19. Analyzing transactions

```
> web3.fromWei(eth.getBalance("0x6d04eA66725A40F5669E2d6f4f77eb4Ecf756a3f"),
"ether")
120010
> web3.eth.getTransaction("0x3cb6cc9cd65c927b0a0af8351e757ef8fdf76ea00d66b88
63b3c9fdcf3657508")
{
  accessList: [],
  blockHash: "0xe772e9613d2f39454f41076dd9e740948bac2e0ecfa254702b4f474fbecc
a438",
  blockNumber: 338,
  chainId: "0x3039",
  from: "0x563a5a670260417b163ad1696fcbfa6849906d06",
  gas: 30000,
  gasPrice: 1000000007,
  hash: "0x3cb6cc9cd65c927b0a0af8351e757ef8fdf76ea00d66b8863b3c9fdcf3657508"
,
  input: "0x",
  maxFeePerGas: 1000000014,
  maxPriorityFeePerGas: 1000000000,
  nonce: 0,
  r: "0x84be672e734bdae2f6d262abda2a08687028a86e035d8c73c8c80662fd6c64fe",
  s: "0x196df54d8f571bde60289c94926793a29f05947203c25418295a1cd80c7f9c6a",
  to: "0x6d04ea66725a40f5669e2d6f4f77eb4ecf756a3f",
  transactionIndex: 0,
  type: "0x2",
  v: "0x0",
  value: 10000000000000000000
}
>
```

Conclusion - Private blockchain was created using geth, the nodes in the blockchain were configured and we interacted with the blockchain by making a transaction of 10 ethers.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 10

Aim: Explore the Cryptocurrency Landscape

Roll No.	54
Name	Arnav Sawant
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

EXPERIMENT 10

Aim: Explore the Cryptocurrency Landscape

Theory:

Cryptocurrency Landscape – Theory

Cryptocurrencies are decentralized digital assets that operate on blockchain technology, enabling secure, transparent, and immutable transactions without the need for a central authority. These systems rely on distributed ledgers where transactions are validated through consensus mechanisms and recorded across a network of nodes.

Data Sources

Reliable and diverse data sources are essential for understanding cryptocurrency markets:

- **Financial Data Platforms** – Services like CoinGecko and CoinMarketCap provide structured data such as market capitalization, circulating supply, trading volume, and historical price trends.
- **News Aggregators** – Platforms such as CryptoPanic collect real-time news from multiple sources, helping track major events impacting the market.
- **Social Sentiment Analytics** – Tools like LunarCrush analyze social media activity to measure public sentiment and engagement.
- **Regulatory Sources** – Official portals like the U.S. Securities and Exchange Commission and frameworks such as MiCA provide insights into legal policies and compliance requirements.

Types of Cryptocurrencies

The cryptocurrency ecosystem consists of various categories based on functionality:

- **Store of Value (SoV)** – Bitcoin (BTC), known for limited supply and long-term value preservation.
- **Utility Tokens** – Ether (ETH) and Chainlink (LINK), used for network operations and smart contract execution.
- **Stablecoins** – Tether (USDT) and USD Coin (USDC), designed to maintain price stability by being pegged to fiat currencies.
- **Governance Tokens** – Uniswap (UNI) and Aave (AAVE), enabling users to participate in decentralized decision-making.

Key Characteristics

Cryptocurrencies exhibit the following features:

- **Decentralization** – No central authority controls the system.

- **Transparency** – Transactions are publicly recorded on the blockchain.
- **Immutability** – Data cannot be altered once confirmed.
- **Security** – Cryptographic algorithms ensure data integrity and safety.

Market Dynamics

The cryptocurrency market is influenced by multiple factors:

- **Volatility** – Prices fluctuate significantly due to demand, sentiment, and external events.
- **Liquidity** – Depends on trading volume and market participation.
- **Market Sentiment** – News, social media, and investor behavior impact price trends.
- **Regulation** – Government policies and frameworks influence adoption and stability.

Blockchain Technology Advancements

Modern blockchain systems are evolving to overcome limitations:

- **Consensus Mechanisms** – Shift from Proof of Work (PoW) to Proof of Stake (PoS) for efficiency.
- **Scalability Solutions** – Layer-2 technologies and sharding improve transaction speed.
- **Interoperability** – Enables communication between different blockchain networks.
- **Privacy Enhancements** – Zero-Knowledge Proofs (ZKP) allow secure verification without revealing sensitive data.

Adoption and Emerging Trends

Cryptocurrency adoption continues to grow across sectors:

- **Digital Payments** – Companies like Stripe and Shopify support crypto transactions.
- **Decentralized Finance (DeFi)** – Financial services operate without intermediaries.
- **Non-Custodial Wallets** – MetaMask and Phantom give users full asset control.
- **Asset Tokenization** – Real-world assets are increasingly represented on blockchain networks.

Utilizing Api for analysis and visualization :

Code:

```
import requests
import matplotlib.pyplot as plt import pandas as pd
import datetime

#      CONFIGURATION
# Replace 'YOUR_API_KEY' with the key you got from CryptoCompare

API_KEY = 'YOUR_API_KEY' SYMBOL = 'LINK' COMPARISON_SYMBOL = 'USD'
LIMIT = 180 # Number of days

# ----- Fetch Data Function -----
def fetch_cryptocompare_data(symbol, comparison_symbol, limit, api_key): url =
f"https://min-api.cryptocompare.com/data/v2/histoday"
params =
{ "fsym":
symbol,
"tsym": comparison_symbol, "limit": limit,
"api_key": api_key
}

response = requests.get(url, params=params) data = response.json()

if data['Response'] == 'Success':
# CryptoCompare returns OHLCV data in 'Data' -> 'Data' return data['Data']['Data']
else:
print(f"Error fetching data: {data.get('Message')}") return None

# ----- Dashboard Function -----
def cryptocompare_dashboard(symbol, comparison_symbol, limit, api_key): raw_data =
fetch_cryptocompare_data(symbol, comparison_symbol, limit,
api_key)
if not raw_data: return

# Create DataFrame
df = pd.DataFrame(raw_data)

# Convert 'time' (unix timestamp) to datetime df['timestamp'] = pd.to_datetime(df['time'], unit='s')
df.set_index('timestamp', inplace=True)
```

```
# Standardize column names to match your previous logic # 'close' is used as the daily price
```

```
df['price'] = df['close']
```

```
df['volume'] = df['volumeto'] # Total volume in USD (volumeto)
```

```
# Calculate metrics
```

```
df['returns'] = df['price'].pct_change() df['MA7'] = df['price'].rolling(window=7).mean() df['MA30']  
= df['price'].rolling(window=30).mean() df['cumulative'] = (1 + df['returns']).cumprod()  
df['volatility'] = df['returns'].rolling(window=7).std() * 100
```

```
# Plotting
```

```
fig, axes = plt.subplots(3, 2, figsize=(16, 12)) fig.suptitle(f'CryptoCompare: {symbol} Analysis  
Dashboard',  
fontsize=16, fontweight='bold')
```

```
# 1. Price Trend
```

```
axes[0, 0].plot(df.index, df['price'], color='tab:blue', linewidth=2) axes[0, 0].set_title('Historical  
Price Trend (USD)')  
axes[0, 0].grid(True, linestyle='--', alpha=0.7)
```

```
# 2. Price vs Volume
```

```
axes[0, 1].plot(df.index, df['price'], color='tab:blue', label='Price')  
axes[0, 1].bar(df.index, df['volume'], color='gray', alpha=0.3, label='Volume (USD)')  
axes[0, 1].set_title('Price vs Trading Volume') axes[0, 1].legend()
```

```
# 3. Daily Returns Distribution
```

```
axes[1, 0].hist(df['returns'].dropna(), bins=40, color='tab:purple', edgecolor='white')  
axes[1, 0].set_title('Daily Returns Distribution')
```

```
# 4. Moving Averages
```

```
axes[1, 1].plot(df.index, df['price'], color='lightgray', alpha=0.5, label='Price')  
axes[1, 1].plot(df.index, df['MA7'], color='tab:orange', label='7-Day MA')  
axes[1, 1].plot(df.index, df['MA30'], color='tab:red', label='30-Day MA')
```

```
axes[1, 1].set_title('Trend Analysis (Moving Averages)') axes[1, 1].legend()
```

```
# 5. Cumulative Returns
```

```
axes[2, 0].fill_between(df.index, df['cumulative'], color='tab:green', alpha=0.2)  
axes[2, 0].plot(df.index, df['cumulative'], color='tab:green') axes[2, 0].set_title('Cumulative  
Growth')
```

6. Volatility

```
axes[2, 1].plot(df.index, df['volatility'], color='tab:red') axes[2, 1].set_title('7-Day Rolling  
Volatility (%)')
```

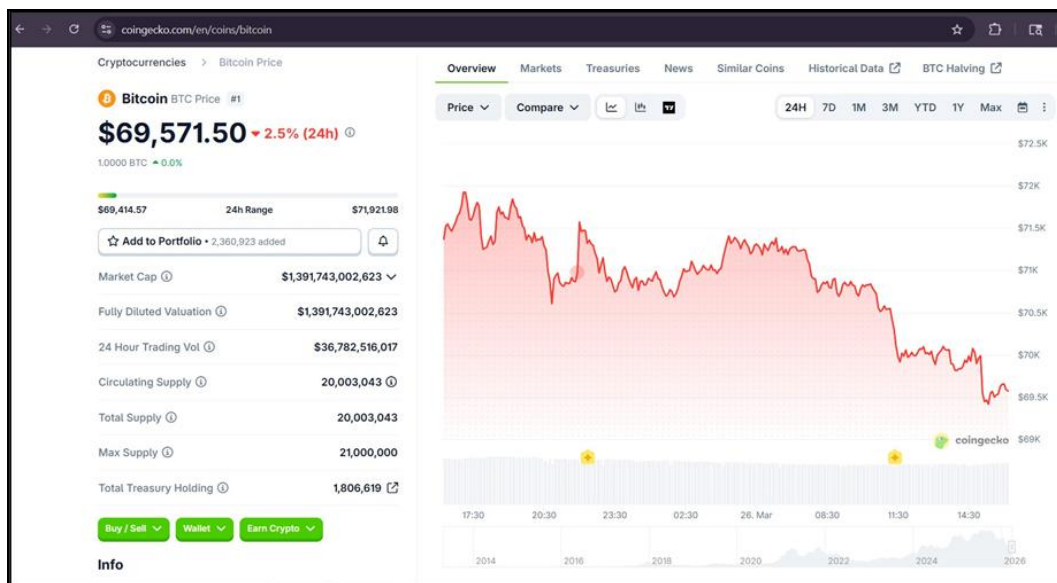
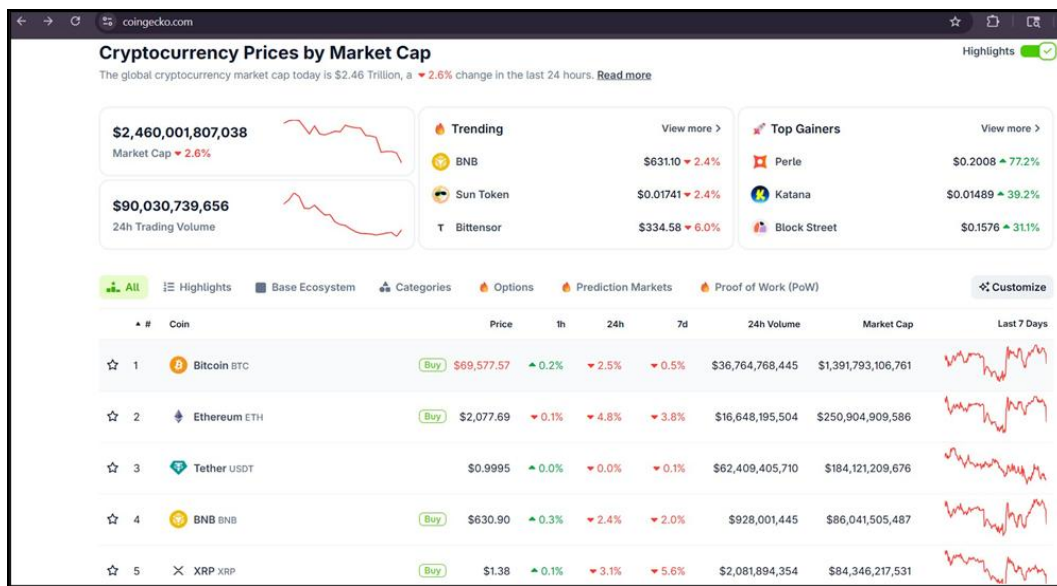
```
plt.tight_layout(rect=[0, 0.03, 1, 0.95]) plt.show()
```

Run

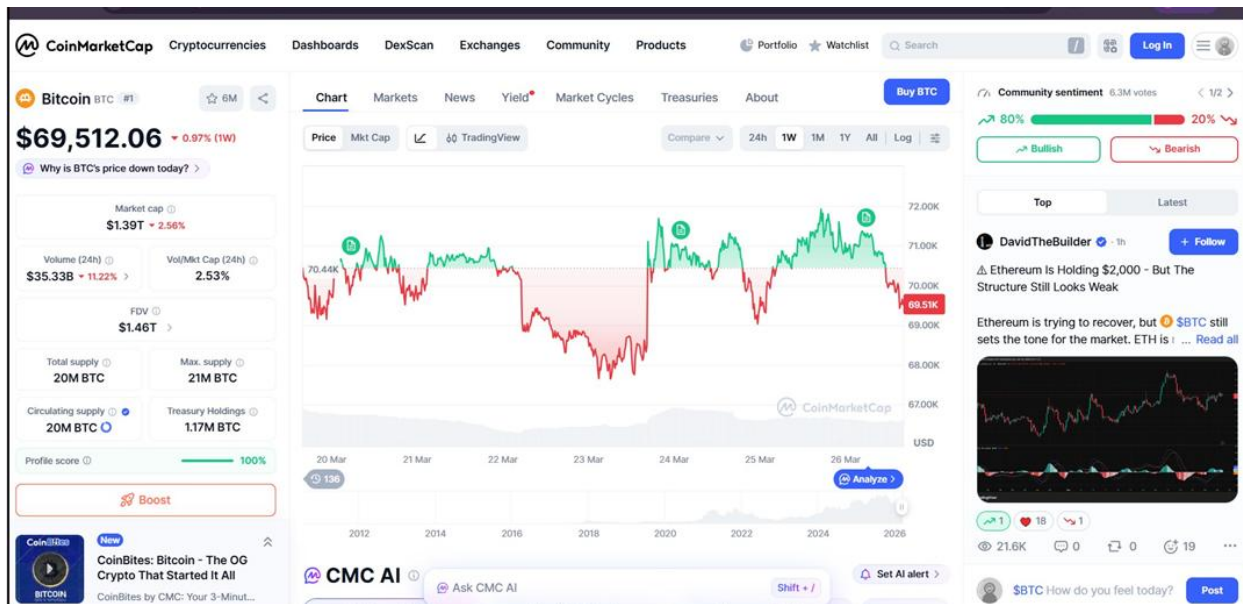
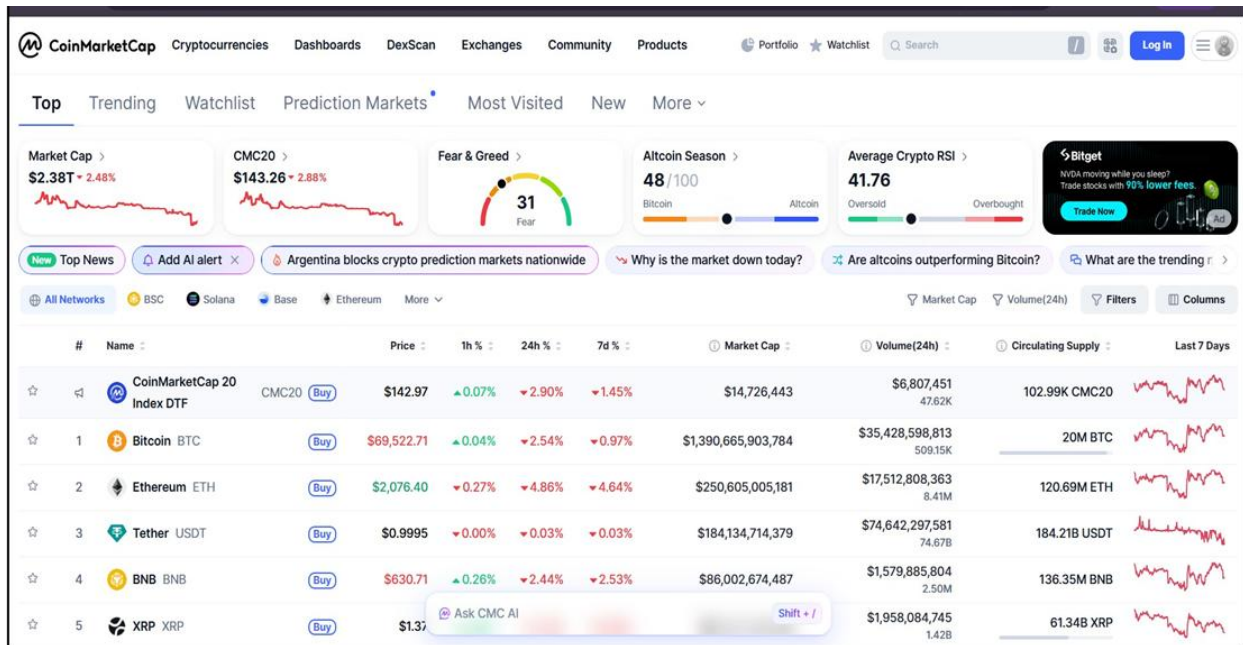
```
cryptocompare_dashboard(SYMBOL, COMPARISON_SYMBOL, LIMIT, API_KEY)
```

Output:-

Coingecko Website

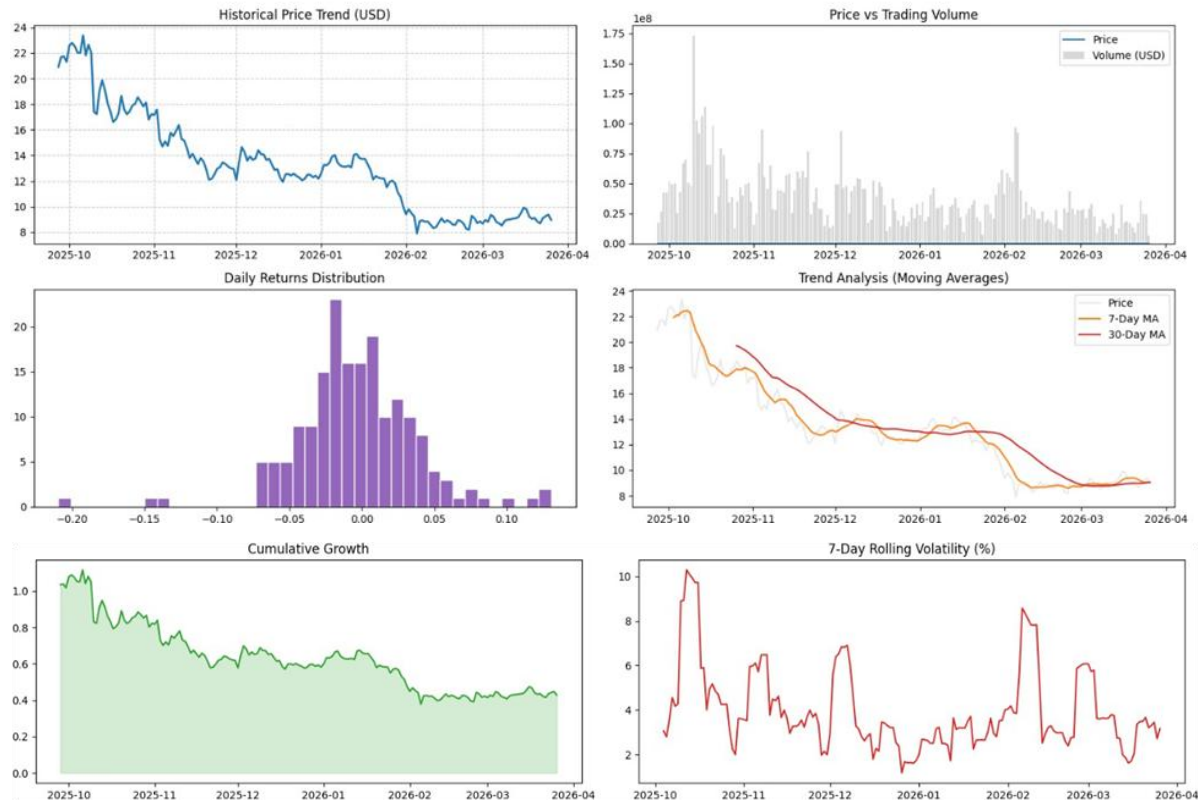


CoinMarketCap website



Dashboard

CryptoCompare: LINK Analysis Dashboard



Conclusion:-

By integrating automated market data processing, visual trend analysis, blockchain technology evaluation, and adoption metrics, a comprehensive view of the cryptocurrency ecosystem can be achieved. This approach highlights the relationship between blockchain's underlying technical foundations and real-world market behavior, offering valuable insights into current trends as well as potential future developments in the digital asset industry.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 11

Aim: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Roll No.	54
Name	Arnav Sawant
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO4:Develop applications in permissioned Hyperledger Fabric network.

Tab 1

EXPERIMENT 11

Aim - Implementation of Hyperledger Fabric Network and Asset Management using Chaincode.

Theory -

Hyperledger Fabric Network Setup

- Consortium & Membership Service Provider (MSP): Fabric is permissioned, meaning only authorized organizations can join. MSPs define identities and roles.
- Peers: Nodes that maintain the ledger and endorse transactions. Each organization typically runs peers.
- Orderer: The ordering service packages endorsed transactions into blocks and delivers them to peers.
- Ledger: Composed of a blockchain (immutable transaction log) and a state database (current values).
- Channels: Private sub-networks where a subset of organizations transact confidentially.

Chaincode (Smart Contracts)

- Chaincode defines the business logic for asset management.
- Written in Go, Java, or Node.js.
- Deployed to peers, then instantiated on a channel.
- Functions typically include createAsset, queryAsset, transferAsset, and deleteAsset.

Transaction Flow

1. Proposal Creation: A client application submits a transaction proposal (e.g., create asset).
2. Endorsement: Peers simulate the transaction using chaincode and return signed results.
3. Ordering: The orderer sequences endorsed transactions into blocks.
4. Validation & Commit: Peers validate endorsements, check for conflicts, and commit the block to the ledger.

Asset Operations

- Asset Creation:
 - Chaincode function createAsset stores asset attributes (ID, owner, value) in the ledger's state database.
- Asset Querying:
 - queryAsset retrieves asset details by key. Rich queries are possible if CouchDB is used as the state database.
- Asset Transfer:
 - transferAsset updates the owner field of an asset. The transaction is endorsed, ordered, and committed, ensuring immutability and consensus.

Key Principles

- Immutability: Once committed, transactions cannot be altered.

- Consensus: Endorsement policies ensure multiple organizations agree before a transaction is valid.
- Privacy: Channels and private data collections allow selective visibility.
- Trust: Identities are cryptographically verified via MSPs.

In essence, Hyperledger Fabric provides a modular, permissioned blockchain framework where chaincode governs asset lifecycles. The theory emphasizes identity-driven trust, modular architecture, and consensus-based validation — ensuring secure creation, querying, and transfer of assets across organizations.

Prerequisite: Check if WSL is Installed

Open **PowerShell (Run as Administrator)** and run:

wsl --list --verbose

If it shows **no distributions**, you must install Ubuntu.

Install Ubuntu for WSL

Run this command in **PowerShell (Admin)**:

wsl --install -d Ubuntu

This installs:

- Ubuntu
- WSL2 kernel

The installation may take a few minutes.

Restart Your Computer

After installation finishes, **restart your PC**.

Open Ubuntu

Open:

Start Menu → Ubuntu

It will ask you to create:

Username

Password

Now Ubuntu is ready.

Implementation

Step 1 — Update the system

code :

sudo apt update

sudo apt upgrade -y

Explain: `apt update` refreshes the package index — it tells your system what versions of software are available. `apt upgrade` installs the newer versions. Fabric's dependencies (Docker, curl, git) can fail or behave unexpectedly on outdated system libraries. Always do this first on a fresh machine.

```
pranav@DESKTOP-OSHSLJI:~$ sudo apt update
sudo apt upgrade -y
[sudo] password for pranav:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1540 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [246 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.6 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [10.6 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [976 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [219 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [20.8 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2683 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [623 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [208 B]
Get:17 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 c-n-f Metadata [544 B]
Get:18 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [28.8 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [6732 B]
Get:20 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:21 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata [396 B]
Get:22 http://archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:23 http://archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:24 http://archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:25 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:26 http://archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:27 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:28 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:29 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1837 kB]
Get:30 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [336 kB]
Get:31 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:32 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.7 kB]
```

Step 2 — Verify prerequisites

code :

```
sudo apt install jq curl git nodejs npm -y
```

```
node -v
```

```
npm -v
```

```
git --version
```

```
docker --version
```

```
docker ps
```

```
pranav@DESKTOP-OSHSLJI:~$ node -v
v18.19.1
pranav@DESKTOP-OSHSLJI:~$ npm -v
11.8.0
pranav@DESKTOP-OSHSLJI:~$ git --version
git version 2.43.0
pranav@DESKTOP-OSHSLJI:~$ docker --version
docker ps
Docker version 29.2.0, build 0b9d198
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

Explain: Fabric requires Docker (to run peer/orderer containers), Git (to clone the repo), and Node.js + npm (to build JavaScript chaincode). Running `docker ps` with no output confirms the Docker daemon is running and your user can access it. If `docker ps` requires sudo, you need to add your user to the docker group.

Step 3 — Clone fabric-samples

code :

```
git clone https://github.com/hyperledger/fabric-samples
```

```
cd fabric-samples
```

```
pranav@DESKTOP-OSHSLJI:~$ git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15527, done.
remote: Counting objects: 100% (98/98), done.
remote: Compressing objects: 100% (60/60), done.
remote: Total 15527 (delta 57), reused 38 (delta 38), pack-reused 15429 (from 3)
Receiving objects: 100% (15527/15527), 24.16 MiB | 6.18 MiB/s, done.
Resolving deltas: 100% (8722/8722), done.
pranav@DESKTOP-OSHSLJI:~$ cd fabric-samples
```

Explain: `fabric-samples` is the official repository containing the test-network scripts, ready-made chaincode examples (including `asset-transfer-basic`), and configuration files. Everything else in this guide runs from inside this directory.

Step 4 — Download the install script and run it

code :

`curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh`

`chmod +x install-fabric.sh`

`./install-fabric.sh docker samples binary`

`./install-fabric.sh docker`

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples$ curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
pranav@DESKTOP-OSHSLJI:~/fabric-samples$ chmod +x install-fabric.sh
pranav@DESKTOP-OSHSLJI:~/fabric-samples$ ./install-fabric.sh docker samples binary

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

====> Downloading version 2.5.15 platform specific fabric binaries
====> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
====> Will unpack to: /home/pranav/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0  0:00:00  0:00:00  0:00:00     0
100 120M  100 120M    0     0 6272k    0  0:00:19  0:00:19  0:00:00 6483k
==> Done.
====> Downloading version 1.5.17 platform specific fabric-client binary
```

```
====> List out hyperledger images
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17          453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15       86ddd7eda92       1GB        255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15     9972c209be47      178MB      49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15        3e25adc31a75      230MB      66.2MB
hyperledger/fabric-baseos:2.5                 654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:2.5.15              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:latest              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-ca:1.5                     453a0a727e82      381MB      123MB
hyperledger/fabric-ca:1.5.17                  453a0a727e82      381MB      123MB
hyperledger/fabric-ca:latest                  453a0a727e82      381MB      123MB
hyperledger/fabric-ccenv:2.5                  86ddd7eda92       1GB        255MB
hyperledger/fabric-ccenv:2.5.15               86ddd7eda92       1GB        255MB
hyperledger/fabric-ccenv:latest               86ddd7eda92       1GB        255MB
hyperledger/fabric-orderer:2.5                9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:2.5.15             9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:latest             9972c209be47      178MB      49.5MB
hyperledger/fabric-peer:2.5                   3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:2.5.15                3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:latest                3e25adc31a75      230MB      66.2MB
```

```

pranav@DESKTOP-OSHSLSJI:~/fabric-samples$ ./install-fabric.sh docker

Pull Hyperledger Fabric docker images

FABRIC_IMAGES: peer orderer ccenv baseos
====> Pulling fabric Images
====> ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
Digest: sha256:3e25adc31a757ee19dd8a24afdc9ac5aa46a86b3cb56fc93068b0e594706d6aa
Status: Image is up to date for ghcr.io/hyperledger/fabric-peer:2.5.15
ghcr.io/hyperledger/fabric-peer:2.5.15
====> ghcr.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
Digest: sha256:9972c209be47f45e377a24c88f2e3d21fae4dc224751c68bdd33f2e7a603ffa8
Status: Image is up to date for ghcr.io/hyperledger/fabric-orderer:2.5.15
ghcr.io/hyperledger/fabric-orderer:2.5.15
====> ghcr.io/hyperledger/fabric-ccenv:2.5.15
2.5.15: Pulling from hyperledger/fabric-ccenv
Digest: sha256:86dded7eda9268e2cbf3691cb952edf545dc5faea1a79bfc9b47ec47d3ecfa9e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ccenv:2.5.15
ghcr.io/hyperledger/fabric-ccenv:2.5.15

```

Explain: This one script does three things simultaneously. **docker** pulls all Fabric Docker images — **fabric-peer**, **fabric-orderer**, **fabric-ccenv**, **fabric-baseos**, **fabric-ca**, and critically **fabric-nodeenv** (needed to build Node.js chaincode). **binary** downloads the CLI tools — **peer**, **configtxgen**, **configtxlator** — into **~/fabric-samples/bin**. **samples** ensures the chaincode sample directories are present. Without **fabric-nodeenv**, chaincode deployment fails later.

Step 5 — Verify Docker images are present

code :

docker images | grep fabric

```

pranav@DESKTOP-OSHSLSJI:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17         453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15       86dded7eda92      1GB        255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15     9972c209be47      178MB      49.5MB      U
ghcr.io/hyperledger/fabric-peer:2.5.15        3e25adc31a75      230MB      66.2MB      U
hyperledger/fabric-baseos:2.5                 654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:2.5.15              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:latest              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-ca:1.5                     453a0a727e82      381MB      123MB
hyperledger/fabric-ca:1.5.17                  453a0a727e82      381MB      123MB
hyperledger/fabric-ca:latest                  453a0a727e82      381MB      123MB
hyperledger/fabric-ccenv:2.5                  86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:2.5.15               86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:latest               86dded7eda92      1GB        255MB
hyperledger/fabric-nodeenv:2.5                17e2d447ca0d      570MB      146MB
hyperledger/fabric-orderer:2.5                9972c209be47      178MB      49.5MB      U
hyperledger/fabric-orderer:2.5.15             9972c209be47      178MB      49.5MB      U
hyperledger/fabric-orderer:latest             9972c209be47      178MB      49.5MB      U
hyperledger/fabric-peer:2.5                   3e25adc31a75      230MB      66.2MB      U
hyperledger/fabric-peer:2.5.15                3e25adc31a75      230MB      66.2MB      U
hyperledger/fabric-peer:latest                3e25adc31a75      230MB      66.2MB      U

```

Explain: Confirm **hyperledger/fabric-nodeenv:2.5** is in the list. This image is the Node.js build environment that Fabric uses to compile and run JavaScript chaincode inside a container. If it is missing, **deployCC** will fail with a "No such image" error.

Step 6 — Set PATH and config path

code :

export PATH=\$PATH:\$HOME/fabric-samples/bin

export FABRIC_CFG_PATH=\$HOME/fabric-samples/config

```

pranav@DESKTOP-OSHSLSJI:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
pranav@DESKTOP-OSHSLSJI:~/fabric-samples$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
pranav@DESKTOP-OSHSLSJI:~/fabric-samples$ peer version
peer:
Version: v2.5.15
Commit SHA: 83c7930
Go version: go1.26.0
OS/Arch: linux/amd64
Chaincode:
Base Docker Label: org.hyperledger.fabric
Docker Namespace: hyperledger

```

Explain: The `peer` binary lives in `~/fabric-samples/bin`, which is not on your shell's PATH by default. Without this export, the shell cannot find the command and returns "peer not found". `FABRIC_CFG_PATH` tells the peer CLI where to find `core.yaml`, the main configuration file. Without it, the peer errors with "cannot init crypto, path does not exist".

code :

peer version

You should see **Version: v2.5.15**.

Step 7 — Start the network and create the channel

code :

cd ~/fabric-samples/test-network

./network.sh up createChannel

```

pranav@DESKTOP-OSHSLSJI:~/fabric-samples/test-network$ cd ~/fabric-samples/test-network
./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
✓Container peer0.org1.example.com      Removed
✓Container orderer.example.com         Removed
✓Container peer0.org2.example.com      Removed
✓Network fabric_test                  Removed
✓Volume compose_orderer.example.com   Removed
✓Volume compose_peer0.org1.example.com Removed
✓Volume compose_peer0.org2.example.com Removed
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59f864cfa1398b52d341cb:latest
Deleted: sha256:27056269ed9759828700d7bfc21149ac76c17d725ae6f9a12c07cc42a04f539f
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59163835f604aa885d60dc:latest
Deleted: sha256:f7c74fe5fd5a6a73117e649e923a5de0a595771f793ee8e1b0047fc35d2d315c

```

```

pranav@DESKTOP-OSHSLJ1:~/fabric-samples/test-network$ ./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using dat
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/pranav/fabric-samples/test-network/./bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] up 7/7
✓Network fabric_test Created
✓Volume compose_orderer.example.com Created
✓Volume compose_peer0.org1.example.com Created
✓Volume compose_peer0.org2.example.com Created
✓Container peer0.org1.example.com Created
✓Container orderer.example.com Created
✓Container peer0.org2.example.com Created
CONTAINER ID IMAGE COMMAND CREATED STATUS

```

Explain: This single command does everything needed to have a running network with a channel. Internally it uses **cryptogen** to generate TLS certificates and identity material (MSP) for Org1, Org2, and the Orderer. It starts three Docker containers — **orderer.example.com** on port 7050, **peer0.org1.example.com** on port 7051, and **peer0.org2.example.com** on port 9051. It then runs **configtxgen** to create **mychannel.block** (the channel genesis block), submits it to the orderer, joins both peers to the channel, and sets anchor peers so the two organizations can discover each other via gossip. The default channel name is **mychannel**.

Step 8 — Deploy the chaincode

code :

```

./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript

```

```

pranav@DESKTOP-OSHSLJ1:~/fabric-samples/test-network$ ./network.sh deployCC \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript \
-ccl javascript
Using docker and docker-compose
Deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1

```

```

2026-03-17 16:28:58.033 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [b424caca912525c6970bb1da73f421e25a00030499eab767783fa7e
atus (VALID) at localhost:7051
2026-03-17 16:28:58.037 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [b424caca912525c6970bb1da73f421e25a00030499eab767783fa7e
atus (VALID) at localhost:9051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required

```

Explain: `-ccn basic` is the name your chaincode will be called by in all future commands. `-ccp` is the path to the chaincode source code. `-ccl javascript` tells Fabric to use the Node.js runtime. Internally this runs the full Fabric chaincode lifecycle — package the source into a `.tar.gz`, install it on both peers, have Org1 approve the definition, have Org2 approve the definition (both approvals are required since majority endorsement is the default policy), and finally commit the definition to the channel. Once committed, two new Docker containers appear — one chaincode container per peer (`dev-peer0.org1...` and `dev-peer0.org2...`).

Step 9 — Set peer identity environment variables

code :

```

export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

```

```

pranav@DESKTOP-OSHSLJ1:~/fabric-samples/test-network$ export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config

export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

```

Explain: The `peer` CLI has no built-in session — it does not know who you are. Every command needs to know which organization's identity to use, which peer to connect to, and how to verify TLS. These five variables provide exactly that. `CORE_PEER_LOCALMSPID` declares "I am Org1". `CORE_PEER_MSPCONFIGPATH` points to the Admin user's private signing key and certificate. `CORE_PEER_TLS_ROOTCERT_FILE` provides the CA certificate to verify the

peer's TLS identity. **CORE_PEER_ADDRESS** is which peer to send requests to. These must be re-exported in every new terminal session.

Step 10 — Initialize the ledger with sample data

code :

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile \  
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/  
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles \  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/  
peer0.org1.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles \  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/  
peer0.org2.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function": "InitLedger", "Args": []}'
```

```
pranav@DESKTOP-0SHSLJ1:~/fabric-samples/test-network$ peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscac  
.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.co  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.co  
--waitForEvent \  
-c '{"function": "InitLedger", "Args": []}'  
2026-03-17 16:37:35.757 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [f982da882feb2411792ddb2c62b47591512c0269f789efe8ab9ce462  
atus (VALID) at localhost:9051  
2026-03-17 16:37:35.757 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [f982da882feb2411792ddb2c62b47591512c0269f789efe8ab9ce462  
atus (VALID) at localhost:7051  
2026-03-17 16:37:35.758 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

Explain: **invoke** is used for any operation that writes to the ledger. **-o localhost:7050** is the orderer address. **--tls** and **--cafile** enable TLS and provide the CA cert to verify the orderer. **-C mychannel** and **-n basic** identify which channel and chaincode. The two **--peerAddresses** and **--tlsRootCertFiles** pairs are the critical part — the default endorsement policy requires a majority of organizations to endorse. With two orgs, both must sign. Sending to only one peer results in **ENDORSEMENT_POLICY_FAILURE**. **--waitForEvent** blocks until both peers confirm the

transaction is committed (status VALID) rather than just submitted. **InitLedger** seeds the ledger with six sample assets.

Step 11 — Query all assets

code :

```
peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["GetAllAssets"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["GetAllAssets"]}'  
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

Explain: **query** is for read-only operations. No transaction is created, no block is written, no endorsement from multiple peers is needed. It is fast and cheap. This returns all six assets seeded by **InitLedger**:

```
[{"ID":"asset1","Color":"blue","Size":5,"Owner":"Tomoko","AppraisedValue":300},  
 {"ID":"asset2","Color":"red","Size":5,"Owner":"Brad","AppraisedValue":400},  
 {"ID":"asset3","Color":"green","Size":10,"Owner":"Jin Soo","AppraisedValue":500},  
 {"ID":"asset4","Color":"yellow","Size":10,"Owner":"Max","AppraisedValue":600},  
 {"ID":"asset5","Color":"black","Size":15,"Owner":"Adriana","AppraisedValue":700},  
 {"ID":"asset6","Color":"white","Size":15,"Owner":"Michel","AppraisedValue":800} ]
```

Step 12 — Read a specific asset

code :

```
peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["ReadAsset","asset5"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'  
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
```

Result:

```
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15}
```

Explain: **ReadAsset** takes an asset ID and returns that single asset's data from the world state. The world state is a key-value database that stores the current (latest) value for each asset. The full transaction history is stored in the blockchain, but queries read from this faster world state.

Step 13 — Create a new asset

code :

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/  
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/  
peer0.org1.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/  
peer0.org2.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function":"CreateAsset","Args":["asset10","blue","10","Arnav","1000"]}'
```

```
pranav@DESKTOP-OSHSLSJI:~/fabric-samples/test-network$ peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tls  
.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example  
--waitForEvent \  
-c '{"function":"CreateAsset","Args":["asset10","blue","10","Pranav","1000"]}'  
2026-03-17 16:38:30.370 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [d88abbf8b7aaff8416fb4a8914a4d01c8064d053ebc5145787034]  
atus (VALID) at localhost:7051  
2026-03-17 16:38:30.371 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [d88abbf8b7aaff8416fb4a8914a4d01c8064d053ebc5145787034]  
atus (VALID) at localhost:9051  
2026-03-17 16:38:30.371 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200  
{"Color":"blue","Size":10,"Owner":"Pranav","AppraisedValue":1000}
```

Explain: **CreateAsset** takes five arguments in order — [ID, Color, Size, Owner, AppraisedValue]. This is a write operation so both peers must endorse. The orderer packages the endorsed transaction into a block and delivers it to both peers. Both confirm VALID and the asset is permanently recorded on the ledger.

code :

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
```

```
pranav@DESKTOP-OSHSLSJI:~/fabric-samples/test-network$ peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["ReadAsset","asset10"]}'  
{"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Pranav","Size":10}  
  
# {"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Arnav","Size":10}
```

The transaction flow showing what happens inside Fabric every time you run **invoke** or **query** Shows what happens inside Fabric every time you run **peer chaincode invoke** — the proposal, endorsement, ordering, and commit flow — and why both peers are always required for any write operation.

Step 14 — Transfer asset ownership

code :

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile  
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/  
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/  
peer0.org1.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/  
peer0.org2.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function": "TransferAsset", "Args": ["asset10", "Rahul"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.c  
.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function": "TransferAsset", "Args": ["asset10", "Rahul"]}'  
2026-03-17 16:41:28.692 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [e834ab1020eac67d6bbf5759291d3c3bd92083d1a340d1f5bd35104c2e387019] committed  
atus (VALID) at localhost:9051  
2026-03-17 16:41:28.692 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [e834ab1020eac67d6bbf5759291d3c3bd92083d1a340d1f5bd35104c2e387019] committed  
atus (VALID) at localhost:7051  
2026-03-17 16:41:28.692 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Pranav"
```

Explain: **TransferAsset** takes [assetID, newOwner]. It reads the current asset from the world state, updates the Owner field, and writes it back. The function returns the previous owner ("Arnav") as confirmation. Both peers endorse, the orderer commits the block, and the ledger is updated permanently.

Verify the ownership change:

code :

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
```

```
# {"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Rahul","Size":10}
```

```
2026-03-17 16:41:28.692 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Pranav"
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
{"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Rahul","Size":10}
```

Owner is now **Rahul**

Step 15 — To stop/bring down the Hyperledger Fabric network, run:

[./network.sh](#) down

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
✓Container orderer.example.com          Removed
✓Container peer0.org2.example.com        Removed
✓Container peer0.org1.example.com        Removed
✓Network fabric_test                    Removed
✓Volume compose_peer0.org1.example.com  Removed
✓Volume compose_peer0.org2.example.com  Removed
✓Volume compose_orderer.example.com     Removed
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035f864cfa1398b52d341cb:latest
Deleted: sha256:aa29d3b9fb04e6316131fbf334a23f28d8a1646fa9d40090916c02d54bc5b76d
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035163835f604aa885d60dc:latest
Deleted: sha256:08d49c295856f950367880744bcacfaf56a7134bf208cacfab8a29ef9221703c
```

Conclusion

The Hyperledger Fabric network was successfully implemented, demonstrating asset creation, querying, and transfer using chaincode. The process highlights key concepts such as consensus, immutability, and secure transaction flow. Overall, it provides a clear understanding of permissioned blockchain systems and their real-world applications in asset management.

Tab 2

Step 1 — Bring down the old network

```
bash
cd ~/fabric-samples/test-network
./network.sh down
```

Step 2 — Set PATH (required every new terminal)

```
bash
export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
```

Step 3 — Start network and create channel

```
bash
cd ~/fabric-samples/test-network
./network.sh up createChannel
```

Step 4 — Deploy chaincode

```
bash
./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
```

Step 5 — Set all environment variables

```
bash
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peer
Organizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrg
anizations/org1.example.com/users/Admin@org1.example.com/msp
```

```
export CORE_PEER_ADDRESS=localhost:7051
```

Step 6 — Initialize the ledger

```
bash
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/
peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/
peer0.org2.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"InitLedger","Args":[]}'
```

Step 7 — Query all assets

```
bash
peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"Args":["GetAllAssets"]}'
```

Step 8 — Read a specific asset

```
bash
peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"Args":["ReadAsset","asset5"]}'
```

Step 9 — Create a new asset

```
bash
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/
peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/
peer0.org2.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"CreateAsset","Args":["asset10","blue","10","Arnav","1000"]}'
```

Verify:

```
bash
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
```

Step 10 — Transfer asset ownership

```
bash
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/
peer0.org1.example.com/tls/ca.crt \
```

```
--peerAddresses localhost:9051 \  
--tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/  
peer0.org2.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function":"TransferAsset","Args":["asset10","Rahul"]}'
```

Verify:

```
bash  
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
```

Step 11 — Stop the network when done

```
bash  
cd ~/fabric-samples/test-network  
./network.sh down
```



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 12

Aim: Mini Project

Roll No.	54
Name	Arnav Sawant
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO4:Develop applications in permissioned Hyperledger Fabric network. CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 13

Aim: Assignment 1

Roll No.	54
Name	Arnav Sawant
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Question : Explore following 4 websites and answer the questions bitcoin.org Blockchain.com https://coinmarketcap.com/ https://www.investopedia.com/
CO Mapped	CO1,CO2

Hard Copy Submitted



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 14

Aim: Assignment 2

Roll No.	54
Name	Arnav Sawant
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Questions ● Draw a neat diagram and explain components of Hyperledger Fabric? ● Draw a neat diagram and explain the transaction flow in Hyperledger Fabric? ● State the requirements of a consensus algorithm. Explain PAXOS and kafka consensus ● Explain ERC 20 token standard and its functions and Compare how ERC 20 token is different than ERC 721 token ● What is Tokenization? Explain its significance and challenges/limitation with example ● Compare Fungible and non Fungible tokens (Give examples of each) ● How can blockchain-based IoT networks improve device management, data monetization, and user privacy in the rapidly growing IoT ecosystem? ● In what ways can blockchain technology enhance transparency, security, and efficiency in government services and record-keeping? Discuss the challenges and benefits of implementing blockchain-based voting systems for elections.
CO Mapped	CO4,CO5,CO6

Hard Copy Submitted



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 15

Aim: Assignment 3

Roll No.	54
Name	Arnav Sawant
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Q1 Analyze & compare different consensus mechanisms such as PoW, PoS, Delegated Proof of Stake & PBFT in terms of security, scalability, energy efficiency & fault tolerance. Illustrate your answer with examples like Bitcoin & Ethereum. Q2 Analyze different types of nodes in a blockchain network & explain their role in maintaining the system. Compare how each node contributes to validation, storage & network efficiency. (Compare UTXO model) Q3 Analyze the UTXO model & explain how it ensures transaction transparency & security in blockchain systems. Illustrate with an example such as Bitcoin. Q4 Explain crypto asset & cryptocurrency. Examine & analyze the role of different types of crypto assets (utility tokens, security tokens, crypto coins) in the blockchain ecosystem. Evaluate their impact on digital finance & decentralized applications. Q5 Analyze the role of payment scripts in blockchain transactions. Examine & analyze different types of payment scripts. Explain how they ensure secure & verifiable transfer of coins. Illustrate with an example. Q6 What is a smart contract? Explain lifecycle of smart contracts. Explain advantages, limitations, risks & opportunities of smart contracts. Q7 Explain the structure of BTC transaction & ETH transaction. Q8 Explain the difference between mining & minting and explain the process
CO Mapped	CO1- CO6

Hard Copy Submitted